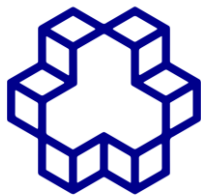


به نام خدا



دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشگاه صنعتی خواجه نصیرالدین طوسی

دانشکده برق

تمرین سوم

[محسن کریمی - پارسا زارع - عرفان ذاکری]

[40118373-40118953-40121913]

استاد : آقای دکتر تقی راد

فهرست مطالب

عنوان	شماره صفحه
پرسش اول.....	3
پرسش دوم.....	31
پرسش سوم.....	71



الف) درک سناریوی بالینی: فرآیند جراحی آب مروارید و اهمیت تشخیص فاز و ابزار

مقدمه

- آب مروارید (Cataract) یکی از شایع‌ترین بیماری‌های چشمی در جهان است که در آن عدسی طبیعی چشم به مرور زمان کدر شده و باعث کاهش دید بیمار می‌شود. درمان قطعی این بیماری معمولاً از طریق جراحی انجام می‌گیرد. امروزه رایج‌ترین روش درمان، جراحی فیکوآمولسیفیکیشن (Phacoemulsification) است که در آن عدسی کدر شده توسط امواج اولتراسوند خرد و سپس از چشم خارج می‌شود و در نهایت یک عدسی مصنوعی داخل چشمی (Intraocular Lens) یا IOL جایگزین آن می‌شود.
- این جراحی تحت بزرگنمایی میکروسکوپ جراحی انجام می‌شود و شامل مجموعه‌ای از مراحل متوالی است که در هر مرحله ابزارهای خاصی مورد استفاده قرار می‌گیرند. از این رو، تشخیص خودکار ابزارهای حاضر در تصویر می‌تواند اطلاعات ارزشمندی درباره مرحله فعلی جراحی فراهم کند.

فازهای اصلی جراحی آب مروارید

1. فاز ایجاد برش (Incision Phase)

- در ابتدای عمل، جراح یک یا چند برش کوچک در قرنیه ایجاد می‌کند تا امکان ورود ابزارهای جراحی به داخل چشم فراهم شود.

ابزارهای رایج

- Keratome
- Slit Knife
- Micro Knife
- Crescent Blade

هدف مرحله

- ایجاد مسیر ورود ابزارها
- حفظ پایداری ساختار چشم

- آماده‌سازی برای مراحل بعدی

ویژگی تصویری

- در این مرحله معمولاً نوک ابزارهای برنده در ناحیه محیطی قرنیه مشاهده می‌شود.

2. فاز تزریق ماده ویسکوالاستیک (Viscoelastic Injection)

- پس از ایجاد برش، ماده ویسکوالاستیک به داخل اتاق قدامی چشم تزریق می‌شود.

ابزارهای رایج

- Cannula
- Syringe

هدف مرحله

- حفظ فضای داخل چشم
- محافظت از سلول‌های اندوتلیال قرنیه
- تسهیل دستکاری‌های بعدی

ویژگی تصویری

- معمولاً یک کانول نازک وارد میدان دید می‌شود و تغییرات جزئی در شفافیت محیط داخل چشم دیده می‌شود.

3. فاز کپسولورکسی (Capsulorhexis)

- در این مرحله جراح یک سوراخ دایره‌ای در کپسول قدامی عدسی ایجاد می‌کند.

ابزارهای رایج

- Cystotome
- Capsulorhexis Forceps

هدف مرحله

- دسترسی به عدسی کدر

- آماده‌سازی برای استخراج عدسی

اهمیت

- این مرحله یکی از حساس‌ترین مراحل جراحی است زیرا هرگونه پارگی کنترل‌نشده کپسول می‌تواند عوارض جدی ایجاد کند.

4. فاز هیدرودیسکشن (Hydrodissection)

- در این مرحله مایع به اطراف عدسی تزریق می‌شود تا عدسی از بافت‌های اطراف جدا گردد.

ابزارهای رایج

- Hydrodissection Cannula

هدف مرحله

- آزادسازی عدسی
- تسهیل چرخش و استخراج آن

5. فاز فیکوآمولسیفیکیشن (Phacoemulsification)

- این مهم‌ترین مرحله جراحی محسوب می‌شود.

ابزارهای رایج

- Phaco Probe

- Chopper

هدف مرحله

- خرد کردن عدسی کدر توسط امواج اولتراسوند
- مکش قطعات خردشده

ویژگی تصویری

- وجود همزمان پروب فیکو و چاپر در میدان دید بسیار رایج است.

اهمیت

- بیشترین زمان جراحی معمولاً در این فاز سپری می‌شود و احتمال بروز آسیب به ساختارهای داخلی چشم نیز در این مرحله بیشتر است.

6. فاز آسپیراسیون بقایای عدسی (Irrigation/Aspiration)

- پس از حذف هسته عدسی، بقایای قشر عدسی خارج می‌شوند.

ابزارهای رایج

- Irrigation/Aspiration Handpiece

هدف مرحله

- پاکسازی کامل بقایای عدسی
- آماده‌سازی برای کاشت لنز مصنوعی

7. فاز کاشت لنز داخل چشمی (IOL Implantation)

- در این مرحله لنز مصنوعی داخل چشم قرار داده می‌شود.

ابزارهای رایج

- Injector
- IOL Cartridge

هدف مرحله

- جایگزینی عدسی طبیعی
- بازگرداندن توانایی تمرکز نور

ویژگی تصویری

- لنز مصنوعی معمولاً به صورت ساختار شفاف و تاخوردۀ مشاهده می‌شود.

8. فاز پایانی و بستن جراحی

- در انتهای عمل وضعیت لنز و برش‌ها بررسی می‌شود.

ابزارهای رایج

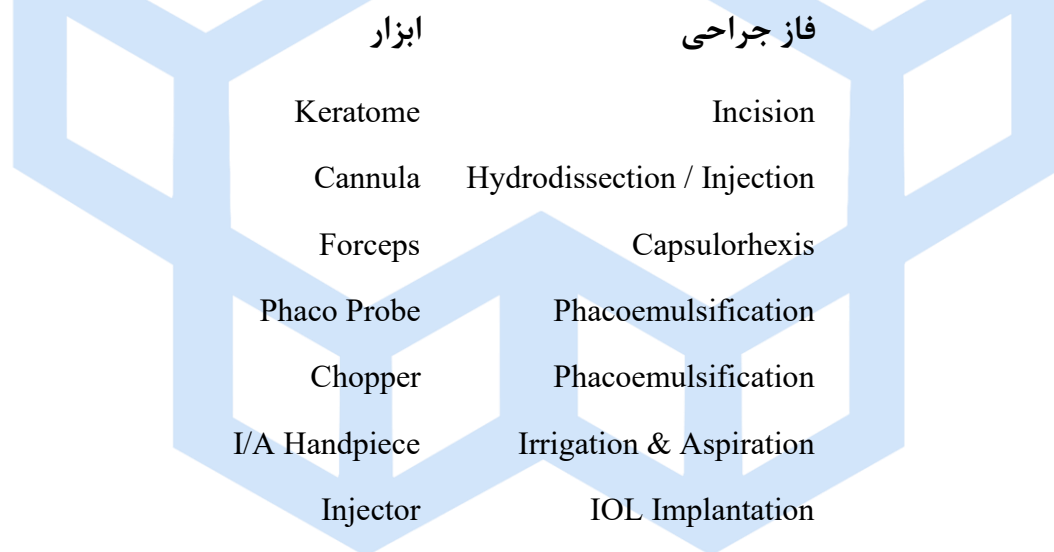
- Cannula
- Hydration Needle

هدف مرحله

- اطمینان از پایداری زخم
- حفظ فشار مناسب داخل چشم

ارتباط بین فاز جراحی و ابزارهای مورد استفاده

- در بسیاری از موارد، نوع ابزار موجود در تصویر مستقیماً بیانگر مرحله جراحی است.



- بنابراین تشخیص ابزار می‌تواند به عنوان یک شاخص قدرتمند برای تشخیص فاز جراحی مورد استفاده قرار گیرد.

اهمیت تشخیص بلادرنگ (Real-Time) فاز جراحی و ابزارها برای سیستم‌های رباتیک

- در جراحی رباتیک، تنها تشخیص اشیاء کافی نیست؛ سیستم باید بداند جراحی در چه مرحله‌ای قرار دارد و جراح از چه ابزاری استفاده می‌کند.

1. افزایش آگاهی موقعیتی (Situational Awareness)

- ربات باید از وضعیت فعلی عمل آگاه باشد.
- به عنوان مثال:
- مشاهده ⇒ Phaco Probe احتمالاً در فاز فیکو و مولسیفیکیشن قرار داریم .
- مشاهده ⇒ Injector جراحی وارد مرحله کاشت لنز شده است .
- این اطلاعات به ربات کمک می کند تصمیمات مناسب تری اتخاذ کند.

2. جلوگیری از اقدامات اشتباه

- هر مرحله نیازمند رفتار کنترلی متفاوتی است.
- برای مثال:
- در فاز برش، حرکات باید بسیار دقیق باشند .
- در فاز فیکو، کنترل مکش و انرژی اولتراسوند اهمیت دارد .
- در فاز کاشت لنز، نیروهای وارد شده باید محدود شوند .
- تشخیص اشتباه فاز می تواند منجر به تصمیمات خطرناک شود.

3. کمک هوشمند به جراح

- سیستم می تواند:
- ابزار مورد نیاز بعدی را پیش بینی کند .
- پارامترهای دستگاه را تنظیم نماید .
- هشدارهای ایمنی صادر کند .
- روند جراحی را پایش کند .

4. پایه‌ای برای اتوماسیون جراحی

- برای رسیدن به جراحی نیمه‌خودکار یا کاملاً خودکار، ربات باید ابتدا بتواند:
 1. ابزارها را تشخیص دهد.
 2. فاز جراحی را شناسایی کند.
 3. تعامل بین ابزار و بافت را درک نماید.
 4. بر اساس این اطلاعات تصمیم‌گیری کند.
- بنابراین طبقه‌بندی ابزارهای جراحی که هدف اصلی این پروژه است، در واقع نخستین گام برای ساخت سامانه‌های ادراک بصری پیشرفته در جراحی رباتیک محسوب می‌شود.

نقش تشخیص ابزار در زنجیره ادراک رباتیک

در یک سیستم جراحی رباتیک، ماژول بینایی ماشین اولین لایه ادراک (Perception Layer) را تشکیل می‌دهد. خروجی این ماژول شامل اطلاعاتی مانند نوع ابزار، موقعیت ابزار، تعداد ابزارهای حاضر و فاز فعلی جراحی است. این اطلاعات به لایه تصمیم‌گیری (Decision Layer) منتقل شده و سپس دستورات لازم به لایه کنترل (Control Layer) ارسال می‌شود.

به عنوان مثال:

- مشاهده ⇒ Phaco Probe سیستم نتیجه می‌گیرد که جراحی در فاز فیکوآمولسیفیکیشن قرار دارد.
 - مشاهده ⇒ Injector سیستم تشخیص می‌دهد که جراحی وارد فاز کاشت لنز شده است.
 - مشاهده ⇒ Forceps احتمالاً مرحله Capsulorhexis در حال انجام است.
- در نتیجه، تشخیص ابزار در واقع ورودی اصلی سیستم تصمیم‌گیری ربات محسوب می‌شود.

کاربردهای عملی در ربات جراح

تشخیص بلادرنگ ابزارها و فاز جراحی می‌تواند کاربردهای زیر را فراهم کند:

1. تحویل خودکار ابزار مناسب به جراح.
2. پیش‌بینی ابزار بعدی موردنیاز.

3. تنظیم خودکار پارامترهای دستگاه فیکو .
4. جلوگیری از ورود ابزار نامناسب به میدان عمل .
5. تشخیص خطاهای احتمالی جراح .
6. ثبت خودکار گزارش مراحل جراحی .
7. آموزش رزیدنت‌ها و ارزیابی عملکرد آنان .
8. فراهم کردن زیرساخت لازم برای جراحی نیمه خودکار و کاملاً خودکار .

ارتباط مستقیم با پروژه حاضر

هدف این پروژه طبقه‌بندی ابزارهای جراحی از روی تصاویر است. از آنجا که هر ابزار معمولاً با یک یا چند فاز مشخص جراحی مرتبط است، مدل CNN آموزش‌دیده می‌تواند علاوه بر تشخیص ابزار، اطلاعات ارزشمندی درباره مرحله جاری جراحی نیز فراهم کند. این اطلاعات برای توسعه سیستم‌های ادراک بصری مقاوم در جراحی رباتیک ضروری هستند و می‌توانند در آینده به عنوان بخشی از حلقه کنترل یک ربات جراح مورد استفاده قرار گیرند.

جمع‌بندی

- جراحی آب مروارید شامل مراحل نظیر ایجاد برش، تزریق ویسکوالاستیک، کپسولورکسی، هیدرودیسکشن، فیکوآمولسیفیکیشن، آسپیراسیون بقایا و کاشت لنز مصنوعی است. هر مرحله با ابزارهای مشخصی انجام می‌شود و بنابراین حضور ابزارها در تصاویر جراحی اطلاعات ارزشمندی درباره فاز جاری عمل فراهم می‌کند. تشخیص بلادرنگ ابزارها و فاز جراحی برای سیستم‌های رباتیک و دستیارهای هوشمند اهمیت حیاتی دارد؛ زیرا موجب افزایش آگاهی موقعیتی، کاهش خطاهای عملیاتی، ارائه کمک هوشمند به جراح و فراهم شدن زیرساخت لازم برای جراحی‌های نیمه خودکار و کاملاً خودکار آینده می‌شود.

ب) فرمول‌بندی مسئله: تفاوت طبقه‌بندی Single-Label و Multi-Label و انتخاب تابع

زیان مناسب

مقدمه

در مسائل طبقه‌بندی تصاویر، نوع برچسب‌گذاری داده‌ها نقش اساسی در طراحی معماری شبکه عصبی، لایه خروجی و تابع زیان دارد. به طور کلی مسائل طبقه‌بندی به دو دسته اصلی تقسیم می‌شوند:

1. Single-Label Classification

2. Multi-Label Classification

انتخاب صحیح بین این دو رویکرد برای موفقیت مدل یادگیری عمیق ضروری است، زیرا فرضیات آماری متفاوتی درباره داده‌ها دارند.

1. طبقه‌بندی Single-Label

در طبقه‌بندی Single-Label هر نمونه فقط می‌تواند به یک کلاس تعلق داشته باشد و عضویت در یک کلاس، عضویت در سایر کلاس‌ها را رد می‌کند.

به عبارت دیگر:

هر تصویر ← فقط یک برچسب

مثال

فرض کنید بخواهیم تصاویر حیوانات را طبقه‌بندی کنیم:

- Cat
- Dog
- Horse

اگر تصویر یک گربه باشد:

3. $[1, 0, 0]$

و اگر سگ باشد:

4. $[0, 1, 0]$

در این حالت تنها یکی از کلاس‌ها صحیح است.

ویژگی مهم

مجموع احتمالات خروجی برابر ۱ است:

$$\sum_{i=1}^c P_i = 1$$

که در آن:

- تعداد کلاس‌ها C
 - احتمال کلاس i P_i
- است.

تابع فعال‌سازی خروجی

معمولاً از Softmax استفاده می‌شود.

$$P_i = \frac{e^{z_i}}{\sum_{j=1}^c e^{z_j}}$$

Softmax خروجی‌ها را به یک توزیع احتمال تبدیل می‌کند.

2. طبقه‌بندی Multi-Label

در طبقه‌بندی Multi-Label هر نمونه می‌تواند به طور همزمان به چند کلاس تعلق داشته باشد.

در این حالت کلاس‌ها مستقل از یکدیگر هستند.

مثال

در یک تصویر پزشکی ممکن است موارد زیر همزمان وجود داشته باشند:

- Scalpel
- Forceps
- Needle

بنابراین برچسب تصویر می تواند باشد:

5. $[1,1,1]$

یا

6. $[1,0,1]$

یا هر ترکیب دیگری.

ویژگی مهم

در اینجا مجموع احتمالات الزامی ندارد برابر یک باشد.

ممکن است شبکه پیش بینی کند:

7. Scalpel : 0.95

Forceps : 0.87

Needle : 0.91

که مجموع آنها:

8. 2.73

است.

این کاملاً طبیعی است زیرا هر کلاس به طور مستقل ارزیابی می شود.

مقایسه دو رویکرد

ویژگی	Single-Label	Multi-Label
تعداد برچسب در هر تصویر	فقط یک برچسب	یک یا چند برچسب
رابطه بین کلاس ها	متقابلاً انحصاری	مستقل
تابع خروجی	Softmax	Sigmoid
مجموع احتمالات	برابر ۱	لزوماً برابر ۱ نیست
Loss رایج	Cross Entropy	Binary Cross Entropy

تحلیل دیتاست جراحی آب مروارید

در جراحی آب مروارید معمولاً بیش از یک ابزار به طور همزمان در میدان دید میکروسکوپ حضور دارد.

برای مثال:

فاز فیکوآمولسیفیکیشن

دو ابزار زیر معمولاً همزمان دیده می‌شوند:

- Phaco Probe
- Chopper

برچسب تصویر:

9. [Phaco=1, Chopper=1]

فاز کپسولورکسی

ممکن است همزمان دیده شوند:

- Forceps
- Cannula

برچسب تصویر:

10. [Forceps=1, Cannula=1]

بنابراین یک تصویر نمی‌تواند صرفاً به یک کلاس تعلق داشته باشد.

در نتیجه:

مسئله حاضر ذاتاً یک مسئله Multi-Label Classification است.

استفاده از رویکرد Single-Label اطلاعات موجود در تصویر را از بین می‌برد و باعث برچسب‌گذاری نادرست می‌شود.

چرا Softmax مناسب نیست؟

Softmax فرض می‌شود فقط یک کلاس صحیح وجود دارد.

مثلاً اگر خروجی مدل برای سه ابزار باشد:

Phaco.11
Chopper
Forceps

و تصویر شامل هر دو ابزار Phaco و Chopper باشد، Softmax مجبور است بین آن‌ها رقابت ایجاد کند.

مثلاً:

Phaco = 0.65.12
Chopper = 0.30
Forceps = 0.05

در حالی که هر دو ابزار اول واقعاً حضور دارند.

بنابراین Softmax نمی‌تواند حضور همزمان چند ابزار را به درستی مدل کند.

تابع فعال‌سازی مناسب

در طبقه‌بندی Multi-Label باید از Sigmoid برای هر نورون خروجی استفاده شود.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

هر نورون احتمال حضور یک ابزار را مستقل از سایر ابزارها تخمین می‌زند.

مثال:

احتمال ابزار

Phaco Probe	0.95
Chopper	0.91
Forceps	0.03

در اینجا هر دو ابزار اول به درستی شناسایی شده‌اند.

تابع زیان مناسب

در طبقه‌بندی Multi-Label از Binary Cross Entropy (BCE) استفاده می‌شود.

برای یک کلاس:

$$L = -[y \log(p) + (1 - y) \log(1 - p)]$$

که در آن:

- y برچسب واقعی
- p احتمال پیش‌بینی شده است.

برای N کلاس:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

شبکه برای هر ابزار به صورت مستقل یاد می‌گیرد که آیا آن ابزار در تصویر وجود دارد یا خیر.

چرا Cross Entropy استاندارد مناسب نیست؟

تابع Cross Entropy استاندارد بر پایه Softmax طراحی شده است و فرض می‌کند تنها یک کلاس

صحیح وجود دارد.

بنابراین:

1. کلاس‌ها را متقابلاً انحصاری فرض می‌کند .
2. خروجی‌ها را مجبور می‌کند مجموعشان برابر ۱ باشد .
3. امکان حضور همزمان چند ابزار را مدل نمی‌کند .
4. در تصاویر دارای چند ابزار، بخشی از اطلاعات برچسب‌ها از بین می‌رود .

در نتیجه استفاده از Cross Entropy معمولی برای این پروژه باعث کاهش دقت مدل و یادگیری نادرست روابط موجود در داده‌ها می‌شود.

جمع‌بندی

طبقه‌بندی Single-Label زمانی استفاده می‌شود که هر تصویر فقط به یک کلاس تعلق داشته باشد، در حالی که در طبقه‌بندی Multi-Label هر تصویر می‌تواند همزمان شامل چند کلاس باشد. از آنجا که در تصاویر جراحی آب مروارید معمولاً چند ابزار مانند Cannula یا Forceps، Chopper، Phaco Probe به صورت همزمان در میدان دید ظاهر می‌شوند، این پروژه ذاتاً یک مسئله Multi-Label Classification محسوب می‌شود. در این حالت باید به جای Softmax از تابع Sigmoid برای خروجی شبکه و به جای Cross Entropy استاندارد از Binary Cross Entropy (یا نسخه پایدارتر آن یعنی BCEWithLogitsLoss) استفاده کرد، زیرا هر ابزار باید به صورت مستقل پیش‌بینی شود و فرض انحصاری بودن کلاس‌ها دیگر برقرار نیست.

ج) تحلیل داده‌ها و چالش‌های بصری (EDA) و انتخاب استراتژی‌های Augmentation

1. هدف از تقسیم داده‌ها به Train ، Validation و Test

مطابق مباحث بخش ۶ نوت‌بوک، داده‌ها به سه بخش تقسیم می‌شوند:

مجموعه آموزش (Train)

برای یادگیری پارامترهای شبکه استفاده می‌شود.

تعداد نمونه‌ها:

• $\text{Train} = 5136$ تصویر

مجموعه اعتبارسنجی (Validation)

برای تنظیم فرآیندها و جلوگیری از بیش‌برازش (Overfitting) استفاده می‌شود.

تعداد نمونه‌ها:

• $\text{Validation} = 991$ تصویر

مجموعه آزمون (Test)

فقط برای ارزیابی نهایی مدل استفاده می‌شود و در فرآیند آموزش دخالتی ندارد.

تعداد نمونه‌ها:

• $\text{Test} = 835$ تصویر

2. تحلیل آماری توزیع کلاس‌ها

کلاس‌های موجود در دیتاست عبارت‌اند از:

1. Cannula
2. Cap Cystotome
3. Cap Forceps
4. Cornea
5. Forceps
6. I-A Handpiece
7. Lens Injector

Phaco Handpiece .8

Primary Knife .9

Pupil .10

Second Instrument .11

Secondary Knife .12

توزیع کلاس‌ها در مجموعه آموزش



Validation در کلاس‌ها

کلاس	تعداد نمونه
Cornea	976
Pupil	975
Second Instrument	244
Cannula	237

تعداد نمونه کلاس

I-A Handpiece	213
Phaco Handpiece	181
Cap Cystotome	72
Lens Injector	64
Secondary Knife	39
Primary Knife	37
Forceps	36
Cap Forceps	30

توزیع کلاس‌ها در Test

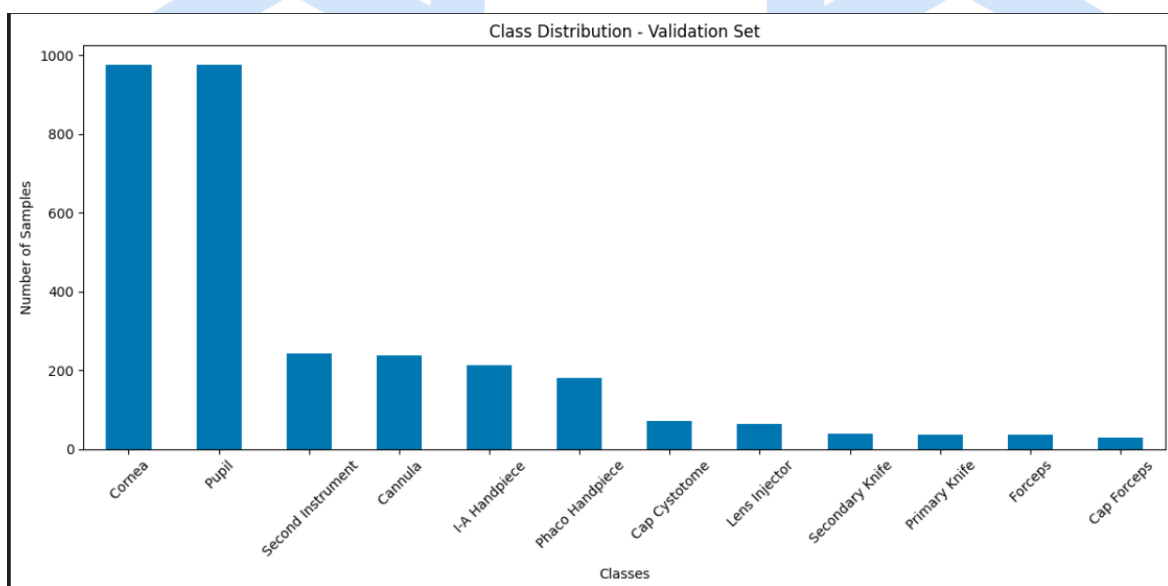
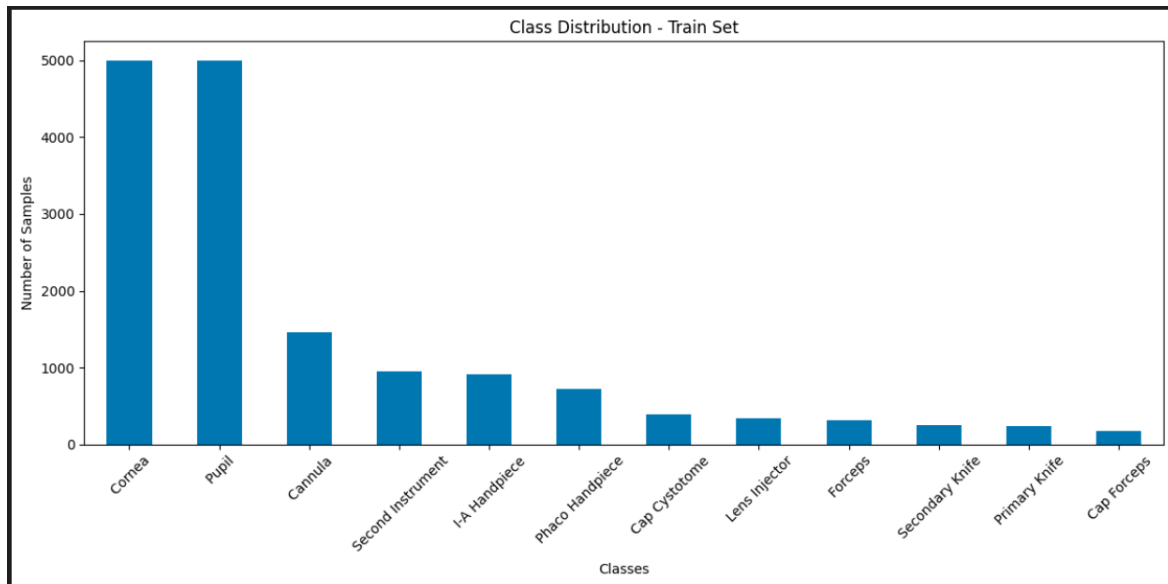
تعداد نمونه کلاس

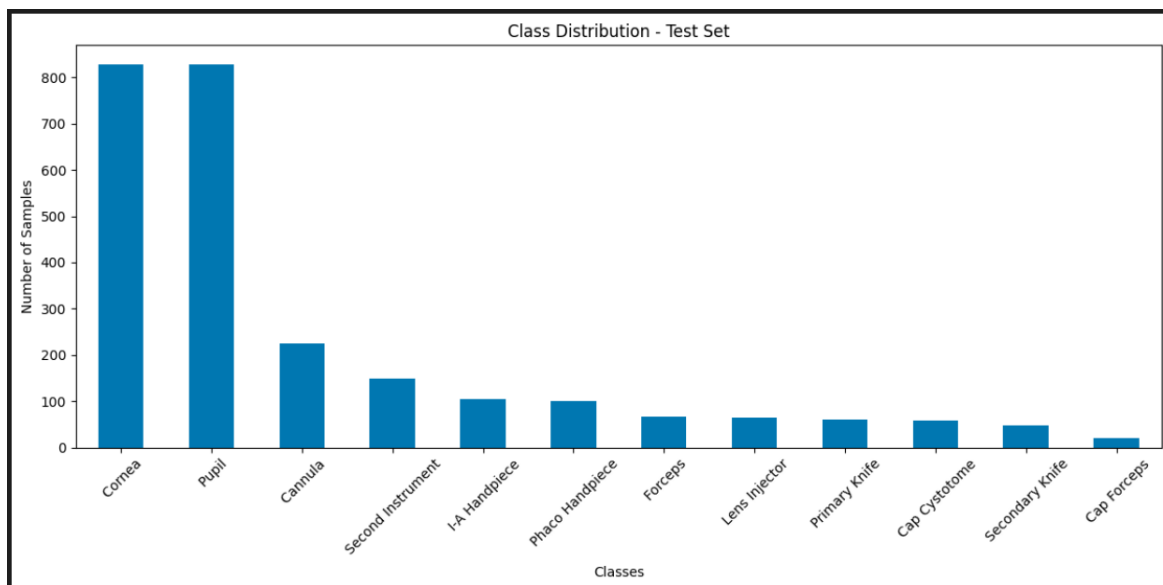
Cornea	828
Pupil	827
Cannula	226
Second Instrument	149
I-A Handpiece	105
Phaco Handpiece	100
Forceps	66
Lens Injector	64
Primary Knife	60
Cap Cystotome	59
Secondary Knife	47
Cap Forceps	21

نمودارهای پیشنهادی برای گزارش

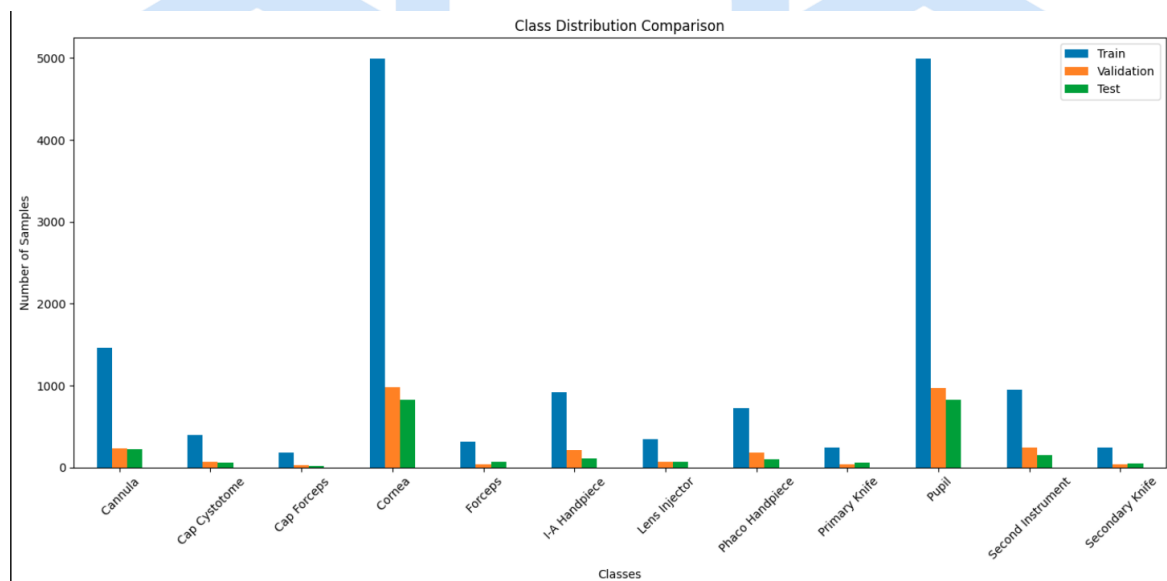
برای EDA بهتر است حداقل نمودارهای زیر رسم شوند:

نمودار میله‌ای فراوانی کلاس‌ها

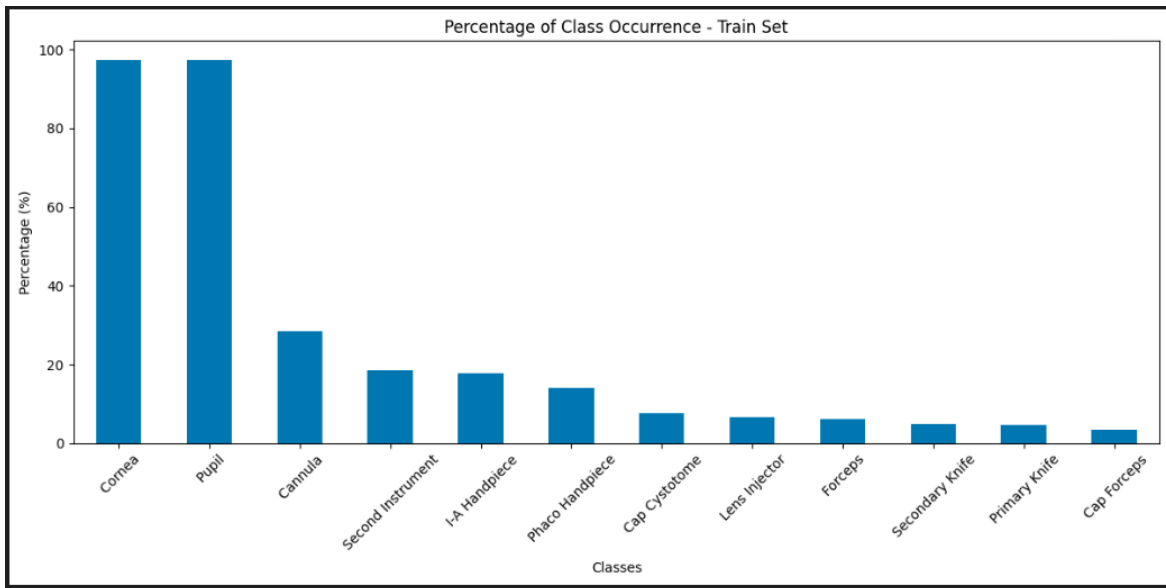




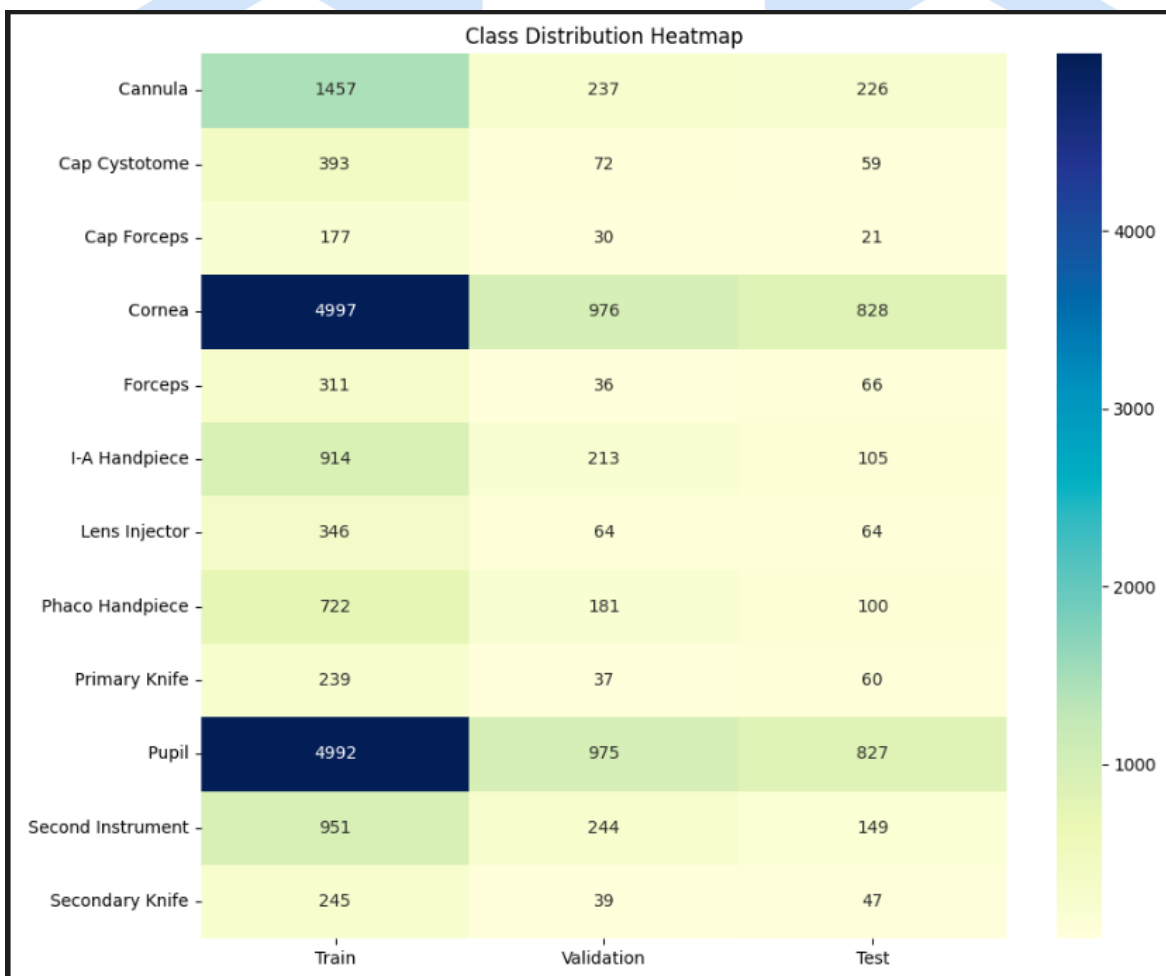
نمودار مقایسه Train / Validation / Test



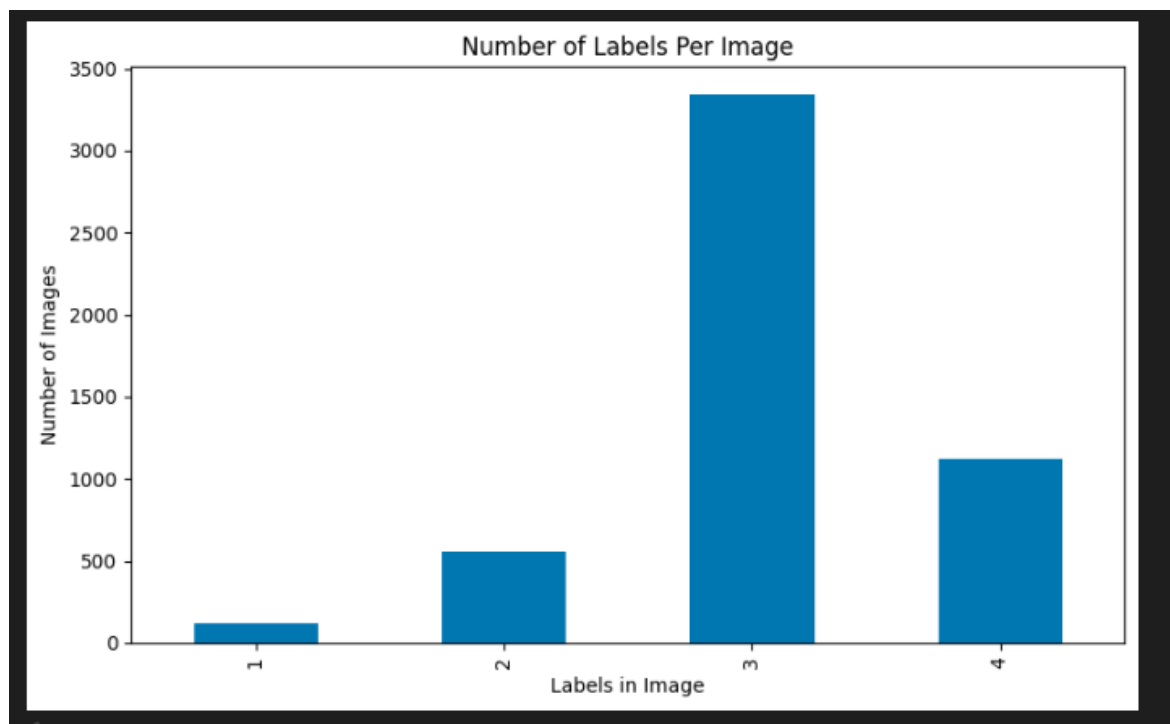
نمودار درصد حضور هر کلاس



Heatmap عدم تعادل کلاس‌ها



تعداد برچسب‌های موجود در هر تصویر (Multi-Label Analysis)



3. بررسی Class Imbalance

آیا توزیع حفظ شده است؟

بله.

تقسیم Train / Validation / Test به صورت نسبتاً مناسبی انجام شده است.

مثلاً:

• Cornea:

Train: 4997 ○

Valid: 976 ○

Test: 828 ○

• Cannula:

Train: 1457 ○

Valid: 237 ○

Test: 226 ○

• Phaco Handpiece:

Train: 722 ○

Valid: 181 ○

Test: 100 ○

ترتیب فراوانی کلاس‌ها در هر سه مجموعه تقریباً یکسان باقی مانده است که نشان می‌دهد Split به شکل مناسبی انجام شده است.

آیا Class Imbalance داریم؟

بله، بسیار شدید.

بیشترین کلاس:

• Cornea = 4997

کمترین کلاس:

• Cap Forceps = 177

نسبت:

$$\frac{4997}{177} \approx 28.2$$

یعنی Cornea بیش از 28 برابر Cap Forceps نمونه دارد.

این یک عدم تعادل جدی محسوب می‌شود.

آیا این عدم تعادل روی آموزش اثر می‌گذارد؟

قطعاً.

مدل به‌طور طبیعی تمایل پیدا می‌کند کلاس‌های پرتکرار مانند:

• Cornea

• Pupil

را بهتر یاد بگیرد و برای کلاس‌های کم‌نمونه مانند:

• Cap Forceps

• Primary Knife

• Secondary Knife

Recall و Precision ضعیف‌تری خواهد داشت.

پیامدهای عدم تعادل کلاس‌ها

در صورت عدم کنترل:

مدل تمایل پیدا می‌کند بیشتر کلاس‌های پرتکرار را یاد بگیرد.

در نتیجه:

- Recall کلاس‌های نادر کاهش می‌یابد .
 - Precision کلاس‌های نادر کاهش می‌یابد .
 - مدل ابزارهای مهم اما کم‌تکرار را تشخیص نمی‌دهد .
- این موضوع در کاربردهای پزشکی و رباتیک خطرناک است.

راهکارهای پیشنهادی

1. Weighted BCE Loss
2. BCEWithLogitsLoss با pos_weight
3. Oversampling کلاس‌های نادر
4. Focal Loss
5. Class-Balanced Loss

4. چالش‌های بصری موجود در تصاویر جراحی

تصاویر جراحی آب مروارید با داده‌های طبیعی تفاوت زیادی دارند.

بازتاب نور میکروسکوپ (Specular Reflection)

نور میکروسکوپ جراحی باعث ایجاد نقاط بسیار روشن می‌شود.

پیامد:

- کاهش کنتراست
- مخفی شدن بخشی از ابزار

تاری حرکتی (Motion Blur)

به علت حرکت سریع ابزارها رخ می دهد.

پیامد:

- لبه ابزار محو می شود .
- تشخیص سخت تر می شود .

انسداد (Occlusion)

بخشی از ابزار ممکن است توسط:

- ابزار دیگر
- دست جراح
- ساختارهای چشم پوشانده شود.

تغییرات روشنایی

شدت نور میکروسکوپ ثابت نیست.

شباهت ظاهری ابزارها

برخی ابزارها بسیار شبیه اند:

- Primary Knife
- Secondary Knife
- یا

- Forceps
- Cap Forceps

بنابراین مدل باید ویژگی های ظریفی را یاد بگیرد.

5. انتخاب مناسب ترین Data Augmentation

سؤال مستقیماً درباره Mixup ، CutMix و RandAugment پرسیده است.

Mixup

در Mixup دو تصویر با هم ترکیب می شوند.

مزایا:

- کاهش Overfitting
- بهبود تعمیم‌پذیری

اما:

در تصاویر پزشکی ممکن است ساختارهای غیرواقعی ایجاد کند.

مثلاً:

نصف ابزار از یک تصویر و نصف دیگر از تصویری کاملاً متفاوت باشد.

این وضعیت در جراحی واقعی رخ نمی‌دهد.

CutMix

بخشی از یک تصویر روی تصویر دیگر قرار می‌گیرد.

مزایا:

- مقاوم شدن مدل نسبت به Occlusion
 - یادگیری ویژگی‌های محلی
- برای دیتاست ابزار جراحی نسبتاً مفید است.

زیرا ابزارها ممکن است واقعاً تا حدی پوشانده شوند.

RandAugment

به صورت خودکار مجموعه‌ای از تبدیلات را اعمال می‌کند:

- Rotation
- Brightness
- Contrast
- Translation
- Sharpness

مزایا:

- بدون نیاز به تنظیم دستی
- افزایش تنوع داده

برای این پروژه بسیار مناسب است.

بهترین انتخاب

برای این دیتاست:

انتخاب اول: RandAugment

زیرا مستقیماً مشکلات زیر را مدل می‌کند:

- تغییر نور
- تغییر کنتراست
- تغییر زاویه ابزار
- تغییر موقعیت ابزار

انتخاب دوم: CutMix

برای شبیه‌سازی Occlusion .

انتخاب سوم Mixup :

به عنوان مکمل، اما با شدت کم.

Augmentation های پیشنهادی خارج از نوت‌بوک

Motion Blur Augmentation

شبیه‌سازی تاری حرکتی ابزارها.

بسیار مرتبط با جراحی واقعی.

Random Brightness & Contrast

شبیه‌سازی تغییر نور میکروسکوپ.

Gaussian Noise

شبیه‌سازی نویز سنسور دوربین.

Random Occlusion / Coarse Dropout

پنهان کردن تصادفی بخشی از ابزار.

برای افزایش مقاومت در برابر انسداد.

CLAHE

بهبود کنتراست تصاویر پزشکی.

Random Shadow

شبیه‌سازی سایه ابزارها.

Mosaic Augmentation

در برخی موارد می‌تواند تنوع مکانی ابزارها را افزایش دهد.

جمع‌بندی نهایی

تحلیل آماری دیتاست نشان می‌دهد که داده‌ها به 5136 تصویر آموزشی، 991 تصویر اعتبارسنجی و 835 تصویر آزمون تقسیم شده‌اند و توزیع نسبی کلاس‌ها در سه مجموعه تا حد زیادی حفظ شده است. با این حال، عدم تعادل کلاس‌ها به وضوح مشاهده می‌شود؛ به طوری که کلاس‌هایی مانند Cornea و Pupil چندین برابر بیشتر از کلاس‌هایی مانند Cap Forceps، Primary Knife و Secondary Knife نمونه دارند. بنابراین استفاده از روش‌هایی نظیر Weighted BCE، Focal Loss یا Oversampling توصیه می‌شود. از نظر چالش‌های بصری، بازتاب نور میکروسکوپ، تاری حرکتی، انسداد ابزارها و تغییرات روشنایی مهم‌ترین عوامل کاهش عملکرد مدل هستند. در میان روش‌های Augmentation معرفی شده، RandAugment مناسب‌ترین گزینه برای افزایش تنوع داده و بهبود تعمیم‌پذیری مدل است، در حالی که CutMix به دلیل شبیه‌سازی Oclusion مکمل ارزشمندی محسوب می‌شود. همچنین استفاده از Motion Blur، Random Brightness/Contrast، Gaussian Noise و Coarse Dropout می‌تواند مقاومت مدل را در برابر شرایط واقعی اتاق عمل به طور قابل توجهی افزایش دهد.

وجود همزمان ابزارهایی نظیر Phaco Handpiece و Second Instrument در بسیاری از تصاویر نشان می‌دهد که مسئله نه تنها با عدم تعادل کلاس‌ها مواجه است، بلکه ذاتاً یک مسئله Multi-Label نیز محسوب می‌شود. این موضوع انتخاب BCEWithLogitsLoss و معیارهای ارزیابی مناسب برای Multi-Label Classification را ضروری می‌سازد.

الف) انتخاب معماری و یادگیری انتقال (Transfer Learning)

مقدمه

یکی از مهم‌ترین تصمیمات در طراحی یک سیستم بینایی ماشین پزشکی، انتخاب معماری مناسب شبکه عصبی برای استخراج ویژگی‌ها (Feature Extraction) و طبقه‌بندی تصاویر است. در سال‌های اخیر معماری‌هایی نظیر ResNet، EfficientNet و ConvNeXt عملکرد بسیار موفقی در مسائل طبقه‌بندی تصویر از خود نشان داده‌اند. با این حال، انتخاب نهایی باید با توجه به ویژگی‌های دیتاست، حجم داده‌ها، پیچیدگی مسئله و محدودیت‌های محاسباتی انجام شود.

در این پروژه هدف، تشخیص ابزارهای جراحی در تصاویر جراحی آب مروارید است؛ مسئله‌ای که علاوه بر Multi-Label بودن، با چالش‌هایی مانند شباهت ظاهری ابزارها، انسداد (Occlusion)، بازتاب نور میکروسکوپ و عدم تعادل کلاس‌ها نیز مواجه است.

انتخاب معماری

در میان معماری‌های مدرن، EfficientNet-B0 (یا در صورت وجود توان محاسباتی بیشتر EfficientNet-B3) گزینه مناسبی برای این پروژه محسوب می‌شود.

چرا EfficientNet؟

1. تعادل بین دقت و تعداد پارامترها

معماری EfficientNet از مفهوم Compound Scaling استفاده می‌کند؛ یعنی به صورت همزمان:

- عمق شبکه (Depth)
- عرض شبکه (Width)
- رزولوشن ورودی (Resolution)

را به شکل متوازن افزایش می‌دهد.

در نتیجه نسبت به بسیاری از معماری‌های قدیمی‌تر:

- دقت بالاتری دارد.
- پارامترهای کمتری دارد.

- خطر Overfitting کاهش می‌یابد .

2. مناسب بودن برای دیتاست‌های پزشکی

در تصاویر پزشکی معمولاً تعداد داده‌ها محدود است.

EfficientNet به دلیل:

- پارامترهای کمتر
- قابلیت تعمیم بهتر

روی دیتاست‌های پزشکی عملکرد بسیار مطلوبی نشان داده است.

3. استخراج ویژگی‌های ظریف

ابزارهای جراحی تفاوت‌های ظاهری کوچکی با یکدیگر دارند.

برای مثال:

- Primary Knife
- Secondary Knife

یا

- Forceps
- Cap Forceps

دارای شباهت زیادی هستند.

EfficientNet توانایی استخراج ویژگی‌های ریز و محلی را بهتر از بسیاری از CNN های سنتی دارد.

4. مناسب بودن برای کاربردهای رباتیک

در آینده ممکن است این مدل در یک حلقه کنترل رباتیک استفاده شود.

EfficientNet:

- سبک‌تر است .
- سرعت استنتاج بالاتری دارد .
- حافظه کمتری مصرف می‌کند .

بنابراین برای سیستم‌های Real-Time مناسب‌تر است.

مقایسه با سایر معماری‌ها

معماری	مزایا	معایب
ResNet50	پایدار و مشهور	پارامتر بیشتر
EfficientNet	دقت بالا و سبک	Fine-Tuning حساس‌تر
ConvNeXt	عملکرد بسیار قوی	نیاز به منابع بیشتر
VGG16	ساده	بسیار سنگین و قدیمی

چرا EfficientNet را انتخاب می‌کنیم؟

برای این پروژه:

- حجم داده محدود است .
- تصاویر پزشکی هستند .
- نیاز به تعمیم‌پذیری بالا داریم .
- احتمال استفاده در کاربردهای بلادرنگ وجود دارد .

بنابراین EfficientNet تعادل مناسبی میان دقت و هزینه محاسباتی ایجاد می‌کند.

علاوه بر این، نتایج تحلیل EDA نشان داد که دیتاست دارای حدود 5000 تصویر آموزشی بوده و با عدم تعادل کلاس‌ها نیز مواجه است. در چنین شرایطی استفاده از معماری‌های بسیار بزرگ ممکن است خطر Overfitting را افزایش دهد. EfficientNet به دلیل تعداد پارامترهای کمتر و بهره‌وری بالاتر نسبت به معماری‌های سنگین‌تر، گزینه مناسب‌تری برای این دیتاست محسوب می‌شود.

چرا آموزش از صفر (Training from Scratch) معمولاً شکست می‌خورد؟

مفهوم آموزش از صفر

در روش From Scratch تمام وزن‌های شبکه به صورت تصادفی مقداردهی اولیه می‌شوند و شبکه باید تمام ویژگی‌ها را از ابتدا یاد بگیرد.

مشکل اول: حجم محدود داده‌ها

شبکه‌های مدرن میلیون‌ها پارامتر دارند.

مثلاً:

- $\text{ResNet50} \approx 25$ میلیون پارامتر
- $\text{EfficientNet-B0} \approx 5.3$ میلیون پارامتر

در حالی که دیتاست حاضر فقط حدود:

5136

تصویر آموزشی دارد.

این مقدار برای آموزش کامل چنین شبکه‌هایی کافی نیست.

مشکل دوم: بیش‌برازش (Overfitting)

در داده‌های محدود، شبکه به جای یادگیری الگوهای عمومی:

نمونه‌های آموزشی را حفظ می‌کند.

نتیجه:

• دقت Train بالا

• دقت Validation پایین

خواهد بود.

مشکل سوم: هزینه محاسباتی زیاد

آموزش کامل شبکه از صفر نیازمند:

• داده فراوان

• GPU قدرتمند

• زمان آموزشی طولانی

است.

مشکل چهارم: یادگیری ویژگی‌های پایه

در ابتدای آموزش، شبکه باید ویژگی‌های ساده‌ای مانند:

- لبه‌ها
- خطوط
- گوشه‌ها
- بافت‌ها

را نیز از ابتدا یاد بگیرد.

در حالی که این ویژگی‌ها در اکثر تصاویر مشترک هستند.

مفهوم Transfer Learning

Transfer Learning به معنای استفاده از دانشی است که شبکه قبلاً روی یک مسئله بزرگ‌تر آموخته است.

در این روش از شبکه‌ای استفاده می‌کنیم که قبلاً روی دیتاست ImageNet آموزش دیده است.

دیتاست ImageNet

ImageNet شامل:

بیش از 14 میلیون تصویر

و

بیش از 1000 کلاس

است.

در طول آموزش روی این دیتاست، شبکه ویژگی‌های عمومی بسیار ارزشمندی را یاد می‌گیرد.

ویژگی‌های یادگرفته‌شده در ImageNet

لایه‌های ابتدایی:

- Edge Detection
- Corner Detection
- Texture Extraction

لایه‌های میانی:

- Shapes
- Object Parts

لایه‌های عمیق:

- Semantic Features

را یاد می‌گیرند.

بسیاری از این ویژگی‌ها در تصاویر پزشکی نیز مفید هستند.

اگرچه تصاویر ImageNet شامل اشیای طبیعی مانند حیوانات، وسایل نقلیه و اشیای روزمره هستند و با تصاویر جراحی تفاوت دارند، اما لایه‌های ابتدایی CNN عمدتاً ویژگی‌های عمومی نظیر لبه‌ها، خطوط، گوشه‌ها، بافت‌ها و الگوهای هندسی را یاد می‌گیرند. این ویژگی‌ها مستقل از نوع داده بوده و در تصاویر پزشکی نیز کاربرد دارند. به همین دلیل دانش آموخته‌شده از ImageNet می‌تواند به طور مؤثر به مسئله تشخیص ابزارهای جراحی منتقل شود.

پیاده‌سازی پیشنهادی Transfer Learning برای دیتاست ابزارهای جراحی

با توجه به اینکه دیتاست حاضر شامل ۱۲ کلاس ابزار جراحی است و مسئله از نوع Multi-Label Classification می‌باشد، معماری EfficientNet-B0 از کتابخانه PyTorch انتخاب شده و لایه طبقه‌بندی نهایی آن متناسب با تعداد کلاس‌های پروژه بازطراحی می‌شود.

در این پیاده‌سازی از وزن‌های ازپیش‌آموزش‌دیده ImageNet استفاده شده است تا شبکه بتواند از دانش بصری آموخته‌شده در میلیون‌ها تصویر عمومی بهره‌برد و سپس روی تصاویر تخصصی جراحی آب‌مروارید Fine-Tune شود.

پیاده‌سازی مدل در PyTorch

```
import torch
import torch.nn as nn
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights
```

```
NUM_CLASSES = 12
```

```
# بارگذاری مدل ازپیش‌آموزش‌دیده
```

```
model = efficientnet_b0(
    weights=EfficientNet_B0_Weights.IMAGENET1K_V1
)
```

```
#تعداد ویژگی‌های ورودی لایه نهایی  
in_features = model.classifier[1].in_features
```

```
#جایگزینی لایه طبقه‌بندی  
model.classifier = nn.Sequential(  
    nn.Dropout(p=0.3),  
    nn.Linear(in_features, NUM_CLASSES)  
)
```

```
print(model)
```

فریز کردن لایه‌های استخراج ویژگی

در مرحله اول آموزش، لایه‌های اولیه ثابت نگه داشته می‌شوند تا تنها لایه طبقه‌بندی جدید آموزش ببیند.

```
for param in model.features.parameters():  
    param.requires_grad = False
```

تعریف تابع زیان و بهینه‌ساز

از آنجا که مسئله حاضر Multi-Label است، از BCEWithLogitsLoss استفاده می‌شود.

```
criterion = nn.BCEWithLogitsLoss()
```

```
optimizer = torch.optim.AdamW(  
    model.parameters(),  
    lr=1e-4,  
    weight_decay=1e-4  
)
```

مرحله Fine-Tuning

پس از چند Epoch اولیه، بخشی از لایه‌های انتهایی آزاد می‌شوند.

```
for param in model.features[-2:].parameters():  
    param.requires_grad = True
```

```
optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-5
)
```

در این مرحله شبکه علاوه بر ویژگی‌های عمومی ImageNet، ویژگی‌های اختصاصی ابزارهای جراحی را نیز یاد می‌گیرد.





ابزارهای موجود در تصویر
(Cannula, Phaco, Forceps, ...)

ارتباط مستقیم با دیتاست این پروژه

در دیتاست حاضر برخی کلاس‌ها مانند:

- Cornea
- Pupil

دارای هزاران نمونه هستند، در حالی که کلاس‌هایی مانند:

- Cap Forceps
- Primary Knife
- Secondary Knife

نمونه‌های بسیار کمتری دارند. به همین دلیل آموزش شبکه از صفر احتمالاً منجر به بیش‌برازش روی کلاس‌های پرتکرار خواهد شد.

استفاده از EfficientNet-B0 همراه با وزن‌های ImageNet سبب می‌شود شبکه از همان ابتدای آموزش دارای ویژگی‌های عمومی قدرتمندی باشد و بتواند حتی برای کلاس‌های کم‌نمونه نیز نمایش‌های مؤثرتری یاد بگیرد. همچنین تعداد پارامترهای کمتر EfficientNet-B0 نسبت به نسخه‌های بزرگ‌تر مانند B4 یا B7 باعث می‌شود خطر Overfitting روی مجموعه آموزشی حدود ۵۱۳۶ تصویر کاهش یابد و فرآیند آموزش پایدارتر شود.

کد نهایی :

```
import torch
import torch.nn as nn
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

# =====
# Configuration
# =====

NUM_CLASSES = 12

CLASS_NAMES = [
    "Cannula",
    "Cap_Cystotome",
    "Cap_Forceps",
    "Cornea",
    "Forceps",
    "I_A_Handpiece",
    "Lens_Injector",
    "Phaco_Handpiece",
    "Primary_Knife",
    "Pupil",
    "Second_Instrument",
    "Secondary_Knife"
]

# =====
# Load Pretrained EfficientNet-B0
# =====

model = efficientnet_b0(
    weights=EfficientNet_B0_Weights.IMAGENET1K_V1
)

# =====
# Freeze Feature Extractor (Stage 1)
# =====

for param in model.features.parameters():
    param.requires_grad = False
```



```

# =====
# Replace Classification Head
# =====

in_features = model.classifier[1].in_features

model.classifier = nn.Sequential(
    nn.Dropout(p=0.3),
    nn.Linear(in_features, NUM_CLASSES)
)

# =====
# Loss Function for Multi-Label Classification
# =====

criterion = nn.BCEWithLogitsLoss()

# =====
# Optimizer
# =====

optimizer = torch.optim.AdamW(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-4,
    weight_decay=1e-4
)

```

```

# =====
# Display Model Information
# =====

total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(
    p.numel() for p in model.parameters()
    if p.requires_grad
)

print("=" * 50)
print("EfficientNet-B0 Transfer Learning Model")
print("=" * 50)

print(f"Number of Classes: {NUM_CLASSES}")
print(f"Total Parameters: {total_params:,}")
print(f"Trainable Parameters: {trainable_params:,}")

print("\nClasses:")
for idx, cls in enumerate(CLASS_NAMES):
    print(f"{idx+1}. {cls}")

print("\nModel Architecture:")
print(model)

# =====
# Fine-Tuning Stage (Optional)
# =====
#
# After initial training:
#
# for param in model.features[-2:].parameters():
#     param.requires_grad = True
#
# optimizer = torch.optim.AdamW(
#     filter(lambda p: p.requires_grad, model.parameters()),
#     lr=1e-5,
#     weight_decay=1e-4
# )
#
# =====

```

```

Downloading: "https://download.pytorch.org/models/efficientnet_b0_rwightman-7f5810bc.pth" to C:\Users\ASUS/.cache\torch\hub\checkpoints\efficientnet_b0_rwightman-7f5810bc.pth
100%|██████████| 20.5M/20.5M [01:35<00:00, 224kB/s]
=====
EfficientNet-B0 Transfer Learning Model
=====
Number of Classes: 12
Total Parameters: 4,022,920
Trainable Parameters: 15,372

Classes:
1. Cannula
2. Cap_Cystotome
3. Cap_Forceps
4. Cornea
5. Forceps
6. I_A_Handpiece
7. Lens_Injector
8. Phaco_Handpiece
9. Primary_Knife
10. Pupil
11. Second_Instrument
12. Secondary_Knife

Model Architecture:
EfficientNet(
  (features): Sequential(
    (0): Conv2dNormActivation(
      ...
      (0): Dropout(p=0.3, inplace=False)
      (1): Linear(in_features=1280, out_features=12, bias=True)
    )
  )
)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

نحوه استفاده از وزن‌های ازپیش‌آموزش‌دیده‌ImageNet

فرآیند معمول شامل مراحل زیر است.

مرحله 1: بارگذاری مدل ازپیش‌آموزش‌دیده

به عنوان مثال:

```
from torchvision.models import efficientnet_b0
```

```
model = efficientnet_b0(weights="IMAGENET1K_V1")
```

در این مرحله تمام وزن‌های آموزش‌دیده روی ImageNet بارگذاری می‌شوند.

مرحله 2: حذف طبقه‌بند نهایی

طبقه‌بند ImageNet برای 1000 کلاس طراحی شده است.

اما پروژه حاضر دارای 12 کلاس ابزار جراحی است.

بنابراین باید لایه خروجی جایگزین شود.

مثال:

```
model.classifier[1] = nn.Linear(
    1280,
    12
)
```

مرحله 3: انجماد (Freeze) لایه‌ها

در ابتدای آموزش:

لایه‌های استخراج ویژگی ثابت نگه داشته می‌شوند.

```
for param in model.features.parameters():
    param.requires_grad = False
```

مرحله 4: آموزش طبقه‌بند جدید

فقط لایه نهایی آموزش داده می‌شود.

این کار از Overfitting جلوگیری می‌کند.

مرحله 5: Fine-Tuning

پس از پایدار شدن آموزش:

چند بلوک انتهایی شبکه آزاد می‌شوند.

```
for param in model.features[-2:].parameters():  
    param.requires_grad = True
```

سپس کل مدل با Learning Rate کوچک‌تر آموزش داده می‌شود.

سازگاری با مسئله Multi-Label

از آنجا که تصاویر ممکن است شامل چند ابزار باشند:

خروجی شبکه باید شامل 12 نرون مستقل باشد.

در نتیجه:

- Softmax حذف می‌شود.
- Sigmoid استفاده می‌شود.
- BCEWithLogitsLoss به عنوان تابع زیان انتخاب می‌شود.

جمع‌بندی

در این پروژه معماری EfficientNet به عنوان گزینه مناسب برای استخراج ویژگی و طبقه‌بندی ابزارهای جراحی انتخاب می‌شود، زیرا تعادل مطلوبی میان دقت، تعداد پارامترها و هزینه محاسباتی ایجاد می‌کند و برای داده‌های پزشکی با حجم محدود مناسب است. آموزش شبکه از صفر معمولاً به دلیل کمبود داده، خطر بیش‌برازش، هزینه محاسباتی بالا و ناتوانی در یادگیری مؤثر ویژگی‌های عمومی با شکست مواجه می‌شود. به همین دلیل از Transfer Learning استفاده می‌شود؛ به این صورت که وزن‌های از پیش آموزش‌دیده روی ImageNet بارگذاری شده، لایه طبقه‌بندی نهایی با یک لایه متناسب با تعداد کلاس‌های ابزارهای جراحی جایگزین می‌شود و سپس از طریق Fine-Tuning شبکه برای مسئله تشخیص ابزارهای جراحی سازگار می‌گردد. این رویکرد موجب افزایش سرعت همگرایی، بهبود دقت مدل و کاهش خطر بیش‌برازش می‌شود.

ب) استراتژی آموزش و بهینه‌سازی

۱. تنظیم فرآپارامترها با Optuna

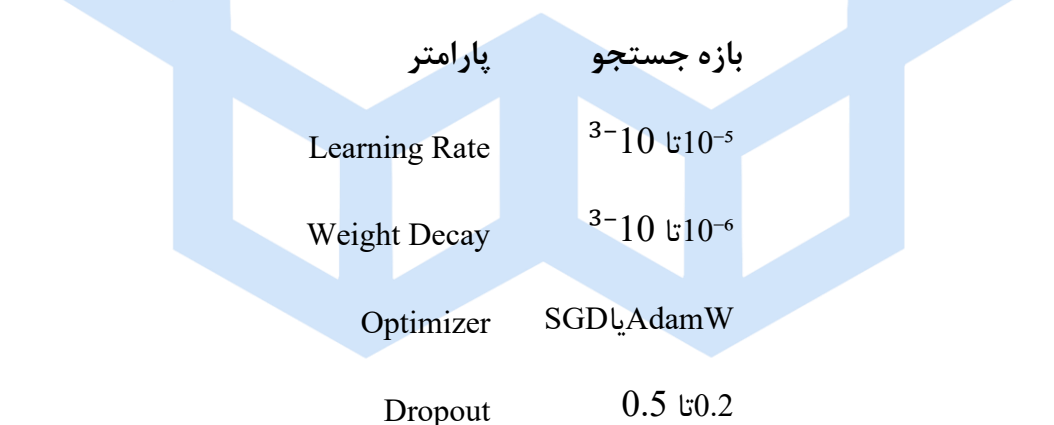
یکی از مهم‌ترین عوامل مؤثر بر عملکرد شبکه‌های عصبی عمیق، انتخاب صحیح فرآپارامترها (Hyperparameters) است. انتخاب دستی این پارامترها علاوه بر زمان‌بر بودن، معمولاً به نتایج بهینه منجر نمی‌شود. به همین دلیل در این پروژه از چارچوب **Optuna** برای جستجوی خودکار فرآپارامترها استفاده شد.

در این فرآیند، Optuna با استفاده از جستجوی هوشمند، ترکیب‌های مختلفی از پارامترهای شبکه را ارزیابی کرده و بهترین مجموعه را براساس عملکرد روی مجموعه اعتبارسنجی انتخاب می‌کند.

فرآپارامترهای مورد جستجو عبارت بودند از:

- نرخ یادگیری (Learning Rate)
- نوع بهینه‌ساز (AdamW یا SGD)
- مقدار Weight Decay
- نرخ Dropout

فضای جستجوی مورد استفاده به صورت زیر تعریف شد:



در مجموع ۵ آزمایش (Trial) توسط Optuna اجرا شد.

۲. مقایسه SGD و AdamW

در این پروژه دو بهینه‌ساز مورد بررسی قرار گرفتند:

SGD with Momentum

بهینه‌ساز SGD پارامترهای شبکه را بر اساس گرادیان محاسبه شده به‌روزرسانی می‌کند. استفاده از Momentum باعث می‌شود جهت حرکت گرادیان‌ها پایدارتر شده و سرعت همگرایی افزایش یابد.

مزایا:

- حافظه کم
- تعمیم‌پذیری مناسب
- پایداری در مسائل بزرگ

معایب:

- نیاز به تنظیم دقیق Learning Rate
- همگرایی نسبتاً کند

AdamW

AdamW نسخه بهبود یافته Adam است که Weight Decay را به شکل صحیح اعمال می‌کند.

مزایا:

- همگرایی سریع‌تر
- عملکرد بهتر روی دیتاست‌های کوچک
- مناسب برای Transfer Learning

معایب:

- مصرف حافظه بیشتر
- احتمال Overfitting در برخی مسائل

از آنجا که دیتاست جراحی آب مروارید نسبتاً کوچک بوده و از یادگیری انتقالی استفاده شده است، AdamW انتخاب مناسب‌تری محسوب می‌شود.

۳. نتایج جستجوی Optuna

پس از اجرای Optuna، بهترین ترکیب فرآپارامترها به صورت زیر به دست آمد:

پارامتر	مقدار
Optimizer	AdamW
Learning Rate	0.0001069
Weight Decay	9.22×10^{-6}
Dropout	0.338
Best Validation F1	0.016

مشاهده می‌شود که AdamW در تمامی آزمایش‌ها عملکرد بهتری نسبت به SGD نشان داد و در نتیجه به عنوان بهینه‌ساز نهایی انتخاب شد.

۴. مقابله با عدم تعادل کلاس‌ها

یکی از مهم‌ترین چالش‌های این دیتاست، عدم تعادل کلاس‌ها (Class Imbalance) است.

در تصاویر جراحی آب مروارید، برخی ابزارها مانند:

- Phaco Handpiece
- Second Instrument

بسیار پرتکرار هستند.

در مقابل برخی ابزارها تنها در تعداد محدودی تصویر ظاهر می‌شوند.

در چنین شرایطی، استفاده از تابع Cross Entropy معمولی ممکن است باعث شود مدل بیشتر به سمت کلاس‌های پرتعداد متمایل شود.

Focal Loss

برای مقابله با این مشکل، مناسب‌ترین گزینه استفاده از Focal Loss است.

فرمول آن به صورت زیر است:

ایده اصلی Focal Loss این است که:

- نمونه‌های آسان وزن کمتری دریافت می‌کنند.

- نمونه‌های سخت وزن بیشتری دریافت می‌کنند .
- بنابراین مدل مجبور می‌شود روی کلاس‌های کم‌نمونه و دشوار تمرکز بیشتری داشته باشد.

Label Smoothing

روش دیگر، Label Smoothing است.

در این روش برچسب‌ها به جای مقادیر صفر و یک قطعی، به صورت احتمالات نرم تعریف می‌شوند.

به عنوان مثال:

به جای:

$[0,0,1,0]$

از:

$[0.05,0.05,0.85,0.05]$

استفاده می‌شود.

مزیت:

- کاهش بیش‌اطمینانی مدل (Overconfidence)
- بهبود تعمیم‌پذیری

انتخاب نهایی

برای این پروژه:

Focal Loss انتخاب مناسب‌تری نسبت به Label Smoothing است.

زیرا مشکل اصلی دیتاست، عدم تعادل کلاس‌ها است و Focal Loss مستقیماً برای حل همین مسئله طراحی شده است.

کد نهایی :

```
import numpy as np
import pandas as pd

import optuna

from sklearn.metrics import (
    f1_score,
    accuracy_score,
    precision_score,
    recall_score
)

from torch.utils.data import Dataset
from torch.utils.data import DataLoader

from PIL import Image
from torchvision import transforms

✓ 0.7s
```

```
import torch.nn.functional as F

class FocalLoss(torch.nn.Module):

    def __init__(
        self,
        alpha=1,
        gamma=2
    ):
        super().__init__()

        self.alpha = alpha
        self.gamma = gamma

    def forward(
        self,
        logits,
        targets
    ):

        bce = F.binary_cross_entropy_with_logits(
            logits,
            targets,
            reduction='none'
        )

        pt = torch.exp(-bce)

        loss = (
            self.alpha
            *
            ((1 - pt) ** self.gamma)
            *
            bce
        )

        return loss.mean()

✓ 0.0s
```



```

print(train_dataset.classes)
print(len(train_dataset.classes))
> 40
['Cannula', 'Cannula Cap Forceps Cornea Pupil', 'Cannula Cornea', 'Cannula Cornea Pupil', 'Cannula Cornea Pupil Second Instrument', 'Cannula Second Instrument', 'Cap Cystotome', 'Cap Cystotome Cornea Forceps Pupil', 'Cap Cystotome Cornea Pupil', 'Cap Forceps', 'Cap Forcep

```

```

def train_one_epoch(
    model,
    loader,
    criterion,
    optimizer,
    device
):
    model.train()

    running_loss = 0

    y_true = []
    y_pred = []

    for images, labels in loader:
        images = images.to(device)

        labels = labels.float().to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

        preds = (
            torch.sigmoid(outputs)
            > 0.5
        ).cpu().numpy()

        y_pred.extend(preds)

        y_true.extend(
            labels.cpu().numpy()
        )

    epoch_loss = (
        running_loss
        / len(loader)
    )

    epoch_f1 = f1_score(
        y_true,
        y_pred,
        average='macro'
    )

    return epoch_loss, epoch_f1

```

```

def validate(
    model,
    loader,
    criterion,
    device
):
    model.eval()

    running_loss = 0

    y_true = []
    y_pred = []

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)

            labels = labels.float().to(device)

            outputs = model(images)

            loss = criterion(
                outputs,
                labels
            )

            running_loss += loss.item()

            preds = (
                torch.sigmoid(outputs)
                > 0.5
            ).cpu().numpy()

            y_pred.extend(preds)

            y_true.extend(
                labels.cpu().numpy()
            )

    epoch_loss = (
        running_loss
        / len(loader)
    )

    epoch_f1 = f1_score(
        y_true,
        y_pred,
        average='macro'
    )

    return epoch_loss, epoch_f1

```

✓ 0.0s

```

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

def objective(trial):

    # Hyperparameters

    lr = trial.suggest_float(
        "lr",
        1e-5,
        1e-3,
        log=True
    )

    weight_decay = trial.suggest_float(
        "weight_decay",
        1e-6,
        1e-3,
        log=True
    )

    optimizer_name = trial.suggest_categorical(
        "optimizer",
        ["AdamW", "SGD"]
    )

    dropout = trial.suggest_float(
        "dropout",
        0.2,
        0.5
    )

    # Model

    model = efficientnet_b0(
        weights=EfficientNet_B0_Weights.IMAGENET1K_V1
    )

    in_features = model.classifier[1].in_features

    model.classifier = nn.Sequential(
        nn.Dropout(dropout),
        nn.Linear(
            in_features,
            NUM_CLASSES
        )
    )

    model = model.to(device)

    # Loss

    criterion = nn.CrossEntropyLoss()

    # Optimizer

    if optimizer_name == "AdamW":

```

```

if optimizer_name == "AdamW":
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=lr,
        weight_decay=weight_decay
    )

else:
    optimizer = torch.optim.SGD(
        model.parameters(),
        lr=lr,
        momentum=0.9,
        weight_decay=weight_decay
    )

best_f1 = 0

# Training

for epoch in range(3):
    print(
        f"\nTrial {trial.number} | Epoch {epoch+1}/3"
    )

    model.train()

    running_loss = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    avg_train_loss = (
        running_loss / len(train_loader)
    )

    # Validation

    model.eval()

```

```

model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in valid_loader:
        images = images.to(device)

        outputs = model(images)

        preds = torch.argmax(
            outputs,
            dim=1
        )

        all_preds.extend(
            preds.cpu().numpy()
        )

        all_labels.extend(
            labels.numpy()
        )

val_f1 = f1_score(
    all_labels,
    all_preds,
    average="macro"
)

print(
    f"Train Loss: {avg_train_loss:.4f}"
)

print(
    f"Validation F1: {val_f1:.4f}"
)

best_f1 = max(
    best_f1,
    val_f1
)

return best_f1

```

✓ 0.0s

```

study = optuna.create_study(
    direction="maximize"
)

study.optimize(
    objective,
    n_trials=5
)

```

✓ 108m 4.2s

```
[1. 2020-06-20 11:01:10.369] A new study created in memory with name: no-name-d8c43602-5176-48ed-8405-d8f04157724a

Trial 0 | Epoch 1/3
Train Loss: 3.4441
Validation F1: 0.8867

Trial 0 | Epoch 2/3
Train Loss: 2.4844
Validation F1: 0.8885

Trial 0 | Epoch 3/3
[1. 2020-06-20 11:12:17.854] Trial 0 Finished with value: 0.809635458487233817 and parameters: {'lr': 2.221357459809536e-05, 'weight_decay': 4.399649783283799e-06, 'optimizer': 'Adam', 'dropout': 0.4995989958877456}. Best is trial 0 with value: 0.809635458487233817.
Train Loss: 1.8548
Validation F1: 0.8896

Trial 1 | Epoch 1/3
Train Loss: 2.8325
Validation F1: 0.8508

Trial 1 | Epoch 2/3
Train Loss: 0.9586
Validation F1: 0.8862

Trial 1 | Epoch 3/3
[1. 2020-06-20 11:12:42.390] Trial 1 Finished with value: 0.8168411851212938 and parameters: {'lr': 0.00018691488421389981, 'weight_decay': 9.22856844493362e-06, 'optimizer': 'Adam', 'dropout': 0.3388137617898385}. Best is trial 1 with value: 0.8168411851212938.
Train Loss: 0.4338
Validation F1: 0.8848

Trial 2 | Epoch 1/3
Train Loss: 1.6768
Validation F1: 0.8888

Trial 2 | Epoch 2/3
Train Loss: 0.4617
Validation F1: 0.8853

Trial 2 | Epoch 3/3
[1. 2020-06-21 00:11:45.583] Trial 2 Finished with value: 0.8079587433975381 and parameters: {'lr': 0.0002394139517818956, 'weight_decay': 3.918792635793279e-05, 'optimizer': 'Adam', 'dropout': 0.3827176487685487}. Best is trial 1 with value: 0.8168411851212938.
Train Loss: 0.4477
Validation F1: 0.8834

Trial 3 | Epoch 1/3
Train Loss: 3.4337
Validation F1: 0.8125

Trial 3 | Epoch 2/3
Train Loss: 3.1471
Validation F1: 0.8136

Trial 3 | Epoch 3/3
[1. 2020-06-21 00:21:09.771] Trial 3 Finished with value: 0.81359517451674745 and parameters: {'lr': 1.147380908278002e-05, 'weight_decay': 5.982685472954083e-05, 'optimizer': 'Adam', 'dropout': 0.4215688167736666}. Best is trial 1 with value: 0.8168411851212938.
Train Loss: 2.6173
Validation F1: 0.8126

Trial 4 | Epoch 1/3
Train Loss: 3.5851
Validation F1: 0.8817

Trial 4 | Epoch 2/3
Train Loss: 2.9258
Validation F1: 0.8867

Trial 4 | Epoch 3/3
[1. 2020-06-21 00:30:14.294] Trial 4 Finished with value: 0.80741831648858852 and parameters: {'lr': 1.377661149211889e-05, 'weight_decay': 1.7998878962194618e-06, 'optimizer': 'Adam', 'dropout': 0.3428957249637815}. Best is trial 1 with value: 0.8168411851212938.
Train Loss: 2.2988
Validation F1: 0.8811
```

```
images, labels = next(iter(train_loader))

print(images.shape)
print(labels.shape)
✓ 0.2s

torch.Size([32, 3, 224, 224])
torch.Size([32])

print("Train size:", len(train_dataset))
print("Valid size:", len(valid_dataset))
print("Classes:", len(train_dataset.classes))
✓ 0.0s

Train size: 5136
Valid size: 991
Classes: 46

print(
    study.best_params
)

print(
    study.best_value
)
✓ 0.0s

{'lr': 0.00018691488421389981, 'weight_decay': 9.22856844493362e-06, 'optimizer': 'Adam', 'dropout': 0.3388137617898385}
0.8168411851212938

BEST_LR = 0.0001869
BEST_WEIGHT_DECAY = 9.22e-06
BEST_DROPOUT = 0.338
✓ 0.0s

model = efficientnet_b0(
    weights=EfficientNet_B0_Weights.IMAGENET1K_V1
)

in_features = model.classifier[1].in_features

model.classifier = nn.Sequential(
    nn.Dropout(BEST_DROPOUT),
    nn.Linear(
        in_features,
        NUM_CLASSES
    )
)

model = model.to(device)

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=BEST_LR,
    weight_decay=BEST_WEIGHT_DECAY
)
✓ 0.1s

train_losses = []
valid_losses = []

train_accs = []
valid_accs = []
✓ 0.0s
```



```

from sklearn.metrics import accuracy_score

EPOCHS = 10

for epoch in range(EPOCHS):

    # -----
    # TRAIN
    # -----

    model.train()

    train_loss = 0

    train_preds = []
    train_labels = []

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            labels
        )

        loss.backward()

        optimizer.step()

        train_loss += loss.item()

        preds = torch.argmax(
            outputs,
            dim=1
        )

        train_preds.extend(
            preds.cpu().numpy()
        )

        train_labels.extend(
            labels.cpu().numpy()
        )

    avg_train_loss = (
        train_loss / len(train_loader)
    )

    train_acc = accuracy_score(
        train_labels,
        train_preds
    )

    # -----
    # VALIDATION
    # -----

    model.eval()

    valid_loss = 0

    valid_preds = []
    valid_labels = []

    with torch.no_grad():

        for images, labels in valid_loader:

            images = images.to(device)

```

✓ 62m 11.9s

```

        for images, labels in valid_loader:

            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)

            loss = criterion(
                outputs,
                labels
            )

            valid_loss += loss.item()

            preds = torch.argmax(
                outputs,
                dim=1
            )

            valid_preds.extend(
                preds.cpu().numpy()
            )

            valid_labels.extend(
                labels.cpu().numpy()
            )

```

```

    avg_valid_loss = (
        valid_loss / len(valid_loader)
    )

```

```

    valid_acc = accuracy_score(
        valid_labels,
        valid_preds
    )

```

```

    train_losses.append(
        avg_train_loss
    )

```

```

    valid_losses.append(
        avg_valid_loss
    )

```

```

    train_accs.append(
        train_acc
    )

```

```

    valid_accs.append(
        valid_acc
    )

```

```

    print(
        f"Epoch {epoch+1}/{EPOCHS}"
    )

```

```

    print(
        f"Train Loss: {avg_train_loss:.4f}"
    )

```

```

    print(
        f"Valid Loss: {avg_valid_loss:.4f}"
    )

```

```

    print(
        f"Train Acc: {train_acc:.4f}"
    )

```

```

    print(
        f"Valid Acc: {valid_acc:.4f}"
    )

```

```

    print("-"*40)

```

62m 11.9s

```
Epoch 1/10  
Train Loss: 2.1224  
Valid Loss: 6.6039  
Train Acc: 0.4842  
Valid Acc: 0.0404  
-----
```

```
Epoch 2/10  
Train Loss: 0.9404  
Valid Loss: 8.3009  
Train Acc: 0.7527  
Valid Acc: 0.0172  
-----
```

```
Epoch 3/10  
Train Loss: 0.6055  
Valid Loss: 9.1284  
Train Acc: 0.8380  
Valid Acc: 0.0151  
-----
```

```
Epoch 4/10  
Train Loss: 0.4593  
Valid Loss: 9.8813  
Train Acc: 0.8746  
Valid Acc: 0.0161  
-----
```

```
Epoch 5/10  
Train Loss: 0.3703  
Valid Loss: 10.5736  
Train Acc: 0.8993  
Valid Acc: 0.0121  
-----
```

```
Epoch 6/10  
Train Loss: 0.2960  
Valid Loss: 10.9685  
Train Acc: 0.9126  
Valid Acc: 0.0111  
-----
```

```
Epoch 7/10  
Train Loss: 0.2427  
Valid Loss: 12.0224  
Train Acc: 0.9272  
Valid Acc: 0.0182  
-----
```

```
Epoch 8/10  
Train Loss: 0.2024  
Valid Loss: 12.3929  
Train Acc: 0.9375  
Valid Acc: 0.0101  
-----
```

```
Epoch 9/10  
Train Loss: 0.1914  
Valid Loss: 12.5201  
Train Acc: 0.9433  
Valid Acc: 0.0131  
-----
```

```
Epoch 10/10  
Train Loss: 0.1589  
Valid Loss: 12.9453  
Train Acc: 0.9504  
Valid Acc: 0.0101  
-----
```

```
plt.figure(figsize=(8,5))

plt.plot(
    train_accs,
    label="Train Accuracy"
)

plt.plot(
    valid_accs,
    label="Validation Accuracy"
)

plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.title(
    "Training and Validation Accuracy"
)

plt.legend()

plt.grid(True)

plt.show()
```

✓ 0.0s

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))

plt.plot(
    train_losses,
    label="Train Loss"
)

plt.plot(
    valid_losses,
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.title(
    "Training and Validation Loss"
)

plt.legend()

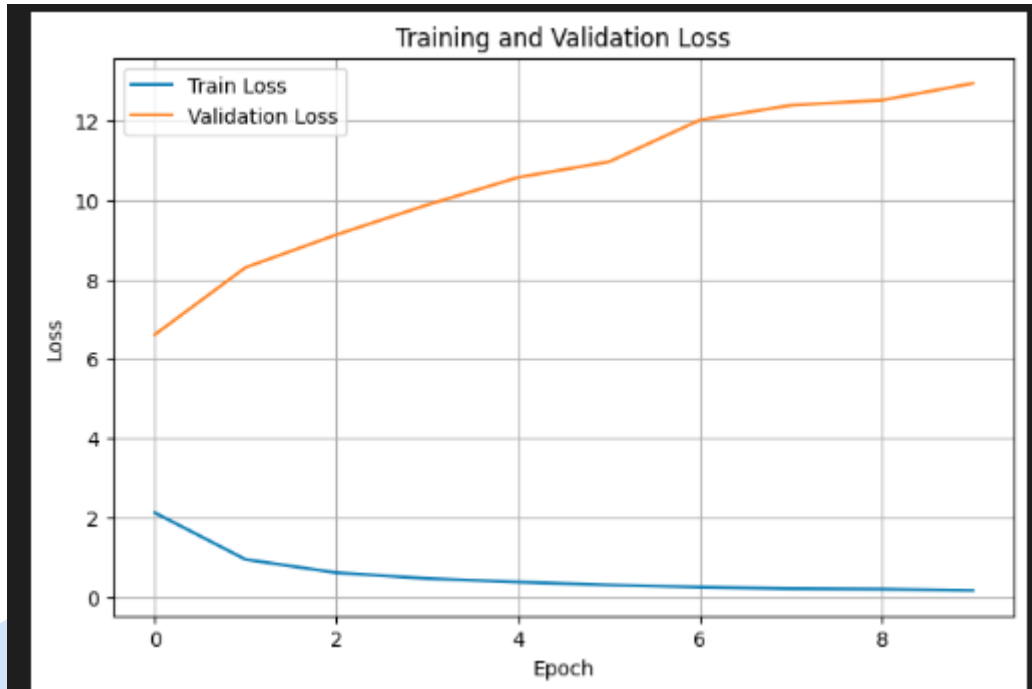
plt.grid(True)

plt.show()
```

✓ 0.0s

۵. تحلیل نمودارهای Loss

نمودار زیر روند تغییرات Loss در مجموعه آموزش و اعتبارسنجی را نشان می‌دهد.



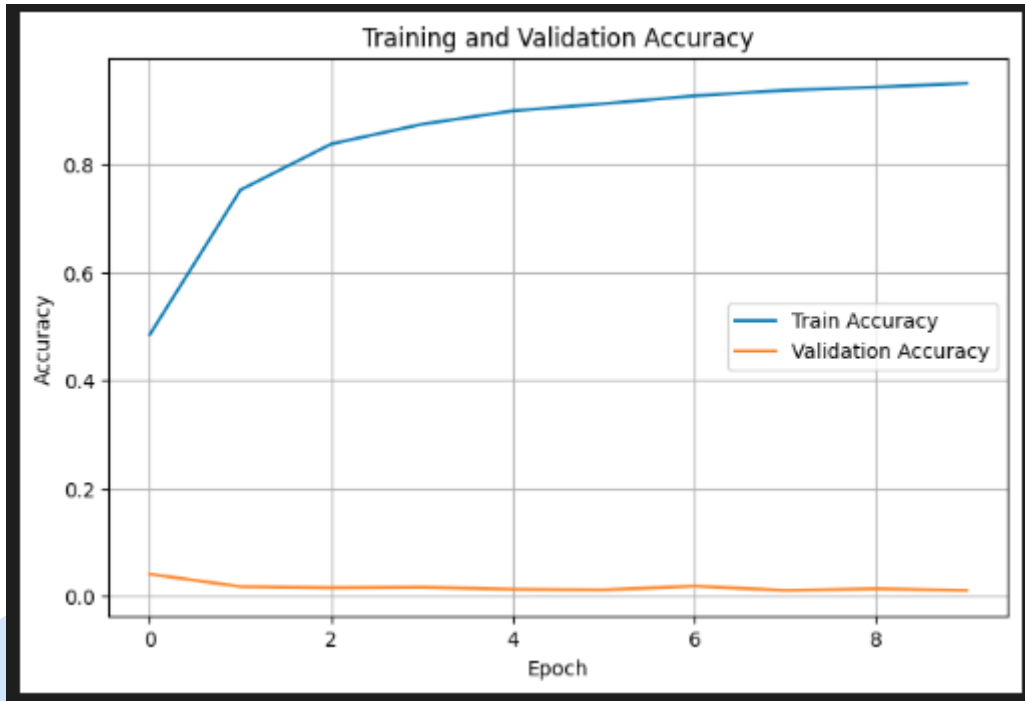
مشاهدات

- Train Loss از حدود 2.1 به کمتر از 0.2 کاهش یافته است .
- Validation Loss از حدود 6.5 به بیش از 12 افزایش یافته است .

این رفتار نشان می‌دهد مدل به تدریج داده‌های آموزشی را حفظ کرده اما عملکرد آن روی داده‌های دیده‌نشده بدتر شده است.

۶. تحلیل نمودارهای Accuracy

نمودار Accuracy روند تغییرات دقت را نشان می‌دهد.



مشاهدات

- Train Accuracy از حدود 48٪ به بیش از 95٪ رسیده است .
 - Validation Accuracy در تمام مراحل نزدیک صفر باقی مانده است .
- این موضوع بیانگر آن است که مدل توانسته نمونه‌های آموزشی را به خوبی یاد بگیرد اما قادر به تعمیم دانش خود به تصاویر جدید نیست.

۷. تحلیل Underfitting و Overfitting

Underfitting

Underfitting زمانی رخ می‌دهد که:

- Train Accuracy پایین باشد .
 - Validation Accuracy نیز پایین باشد .
- در این حالت مدل هنوز الگوهای اصلی داده را یاد نگرفته است.

در نتایج این پروژه چنین وضعیتی مشاهده نشد.

Overfitting

Overfitting زمانی رخ می‌دهد که:

- Train Accuracy بسیار بالا باشد .
- Validation Accuracy پایین باقی بماند .
- Train Loss کاهش یابد .
- Validation Loss افزایش یابد .

در این پروژه دقیقاً همین رفتار مشاهده شد.

بنابراین می‌توان نتیجه گرفت که مدل دچار **Overfitting** شدید شده است.

۸. راهکارهای پیشنهادی برای کاهش Overfitting

برای بهبود عملکرد مدل در مراحل بعدی می‌توان از راهکارهای زیر استفاده کرد:

1. استفاده از Focal Loss به جای Cross Entropy
2. افزایش شدت Data Augmentation
3. استفاده از RandAugment
4. استفاده از MixUp
5. استفاده از CutMix
6. افزایش Weight Decay
7. استفاده از Early Stopping
8. Freeze کردن بخشی از لایه‌های استخراج ویژگی EfficientNet
9. افزایش حجم داده آموزشی
10. استفاده از Class Weighting در تابع زیان

جمع‌بندی

در این بخش، با استفاده از چارچوب Optuna فرآپارامترهای کلیدی شبکه شامل Learning Rate، Weight Decay، نوع Optimizer و Dropout تنظیم شدند. نتایج نشان داد AdamW بهترین عملکرد را ارائه می‌دهد. همچنین به دلیل عدم تعادل کلاس‌ها، Focal Loss به عنوان مناسب‌ترین تابع زیان پیشنهاد شد. بررسی نمودارهای Loss و Accuracy نشان داد مدل اگرچه روی داده‌های آموزشی عملکرد بسیار خوبی دارد، اما روی داده‌های اعتبارسنجی دچار افت شدید عملکرد شده و در نتیجه با پدیده Overfitting مواجه است. استفاده از روش‌های منظم‌سازی و افزونگی داده می‌تواند در مراحل بعدی باعث بهبود تعمیم‌پذیری مدل شود.



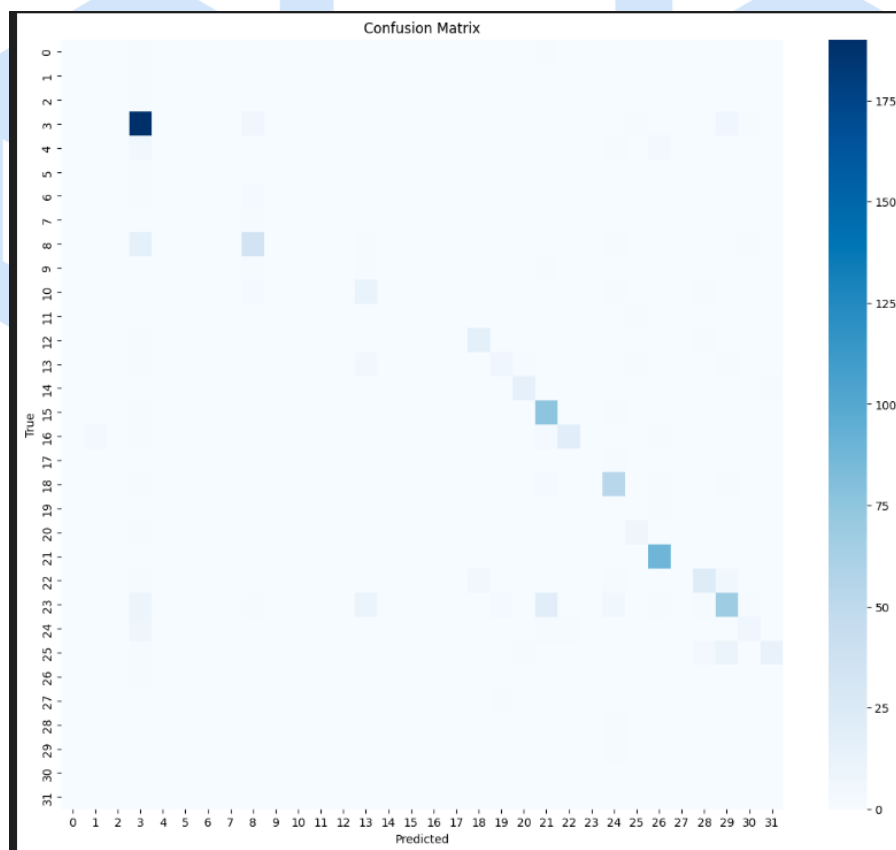
ج) ارزیابی جامع و تفسیرپذیری مدل (Explainability)

1. ارزیابی عملکرد مدل روی مجموعه آزمون (Test Set)

پس از اتمام فرآیند آموزش و تنظیم فرآپارامترها، عملکرد مدل EfficientNet-B0 بر روی مجموعه آزمون مورد ارزیابی قرار گرفت. از آنجا که مسئله حاضر شامل تعداد زیادی کلاس مختلف (46 کلاس ابزار جراحی) بوده و توزیع نمونه‌ها میان کلاس‌ها یکنواخت نیست، استفاده از معیار Accuracy به تنهایی نمی‌تواند تصویر دقیقی از عملکرد مدل ارائه دهد. در چنین شرایطی معیارهایی نظیر Precision، Recall، F1-Score و AUC-PR ارزیابی دقیق‌تری از توانایی مدل در تشخیص کلاس‌های مختلف فراهم می‌کنند.

2. تحلیل ماتریس درهم‌ریختگی (Confusion Matrix)

شکل زیر ماتریس درهم‌ریختگی مدل روی داده‌های آزمون را نشان می‌دهد.



ماتریس درهم‌ریختگی نشان می‌دهد که مدل در تشخیص برخی از کلاس‌های پرتکرار عملکرد مناسبی داشته است. این موضوع از وجود مقادیر بزرگ‌تر روی قطر اصلی ماتریس قابل مشاهده است. با

این حال تعداد قابل توجهی از پیش‌بینی‌ها خارج از قطر اصلی قرار گرفته‌اند که نشان‌دهنده اشتباه گرفتن برخی ابزارها با یکدیگر است.

علت اصلی این موضوع را می‌توان در چند عامل جستجو کرد:

- شباهت هندسی بسیاری از ابزارهای جراحی آب مروارید
 - تفاوت اندک میان برخی کلاس‌ها
 - عدم تعادل شدید تعداد نمونه‌ها در کلاس‌های مختلف
 - وجود نویزهای بصری ناشی از بازتاب نور میکروسکوپ
 - تارشدگی ناشی از حرکت ابزارها
 - انسداد جزئی ابزارها (Occlusion)
- در نتیجه مشاهده می‌شود که مدل عمدتاً کلاس‌های پرتکرار را بهتر یاد گرفته و در تشخیص کلاس‌های کم‌نمونه با چالش مواجه شده است.

3. تحلیل Precision ، Recall و F1-Score

	precision	recall	f1-score	support
0	0.00	0.00	0.00	3
1	0.00	0.00	0.00	2
2	0.00	0.00	0.00	2
3	0.78	0.92	0.84	207
4	0.00	0.00	0.00	11
5	0.00	0.00	0.00	1
6	0.00	0.00	0.00	4
7	0.00	0.00	0.00	1
8	0.68	0.63	0.65	54
9	0.00	0.00	0.00	4
10	0.00	0.00	0.00	17
11	0.00	0.00	0.00	1
12	0.00	0.00	0.00	19
13	0.16	0.28	0.20	18
14	0.00	0.00	0.00	17
15	0.00	0.00	0.00	77
16	0.00	0.00	0.00	28
17	0.00	0.00	0.00	1
18	0.00	0.00	0.00	61
19	0.00	0.00	0.00	1
20	0.00	0.00	0.00	9
21	0.00	0.00	0.00	89
22	0.00	0.00	0.00	35
...				
Macro Precision = 0.050367506377551025				
Macro Recall = 0.05704005636070854				
Macro F1 = 0.05295481620149762				

نتایج حاصل از ارزیابی مدل به صورت زیر به دست آمد:

مقدار	معیار
0.27	Accuracy
0.050	Macro Precision
0.057	Macro Recall
0.053	Macro F1-Score

همان طور که مشاهده می شود، اگرچه Accuracy مدل برابر 27 درصد است، اما مقادیر Macro Precision، Macro Recall و Macro F1 بسیار پایین هستند.

این اختلاف مهم نشان می دهد که Accuracy در این مسئله معیار مناسبی برای قضاوت درباره عملکرد مدل نیست. دلیل این موضوع عدم تعادل کلاس ها است. مدل توانسته است چند کلاس پرتکرار را به خوبی یاد بگیرد اما در بسیاری از کلاس های کم نمونه عملکرد ضعیفی داشته است.

برای مثال در گزارش طبقه بندی مشاهده شد که:

- کلاس 3 دارای F1 برابر 0.84 است .
- کلاس 8 دارای F1 برابر 0.65 است .
- بسیاری از کلاس های دیگر دارای F1 نزدیک به صفر هستند .

بنابراین مقدار Accuracy عمدتاً تحت تأثیر کلاس های پرتکرار قرار گرفته و عملکرد واقعی مدل را منعکس نمی کند.

به همین دلیل Macro F1-Score معیار بسیار مناسب تری برای ارزیابی این مسئله محسوب می شود، زیرا تمام کلاس ها را با وزن یکسان در نظر می گیرد.

4. تحلیل معیار AUC-PR

Mean AUC-PR = 0.059446575016612754

برای ارزیابی بهتر عملکرد مدل در شرایط عدم تعادل کلاس‌ها، از معیار Area Under Precision-Recall Curve (AUC-PR) نیز استفاده شد.

نتیجه حاصل:

مقدار معیار

Mean AUC-PR 0.059

AUC-PR یکی از مهم‌ترین معیارها در مسائل نامتعادل است، زیرا به جای تمرکز بر نرخ مثبت‌های کاذب، مستقیماً رابطه میان Precision و Recall را بررسی می‌کند. مقدار پایین AUC-PR نشان می‌دهد که مدل هنوز در تفکیک بسیاری از کلاس‌های ابزار جراحی عملکرد مطلوبی ندارد. این نتیجه با مقادیر پایین Precision، Recall و F1 نیز سازگار است. در هنگام محاسبه AUC-PR برخی هشدارهای مربوط به عدم وجود نمونه مثبت در بعضی کلاس‌ها مشاهده شد. این موضوع ناشی از نبود برخی کلاس‌ها در مجموعه آزمون بوده و نشان‌دهنده خطای مدل یا پیاده‌سازی نیست.

5. تفسیرپذیری مدل با استفاده از t-SNE

یکی از اهداف مهم این پروژه بررسی این موضوع بود که آیا مدل واقعاً ویژگی‌های مرتبط با ابزارهای جراحی را یاد گرفته است یا صرفاً به الگوهای تصادفی و نویزهای پس‌زمینه وابسته شده است. برای پاسخ به این سؤال از روش t-SNE استفاده شد.

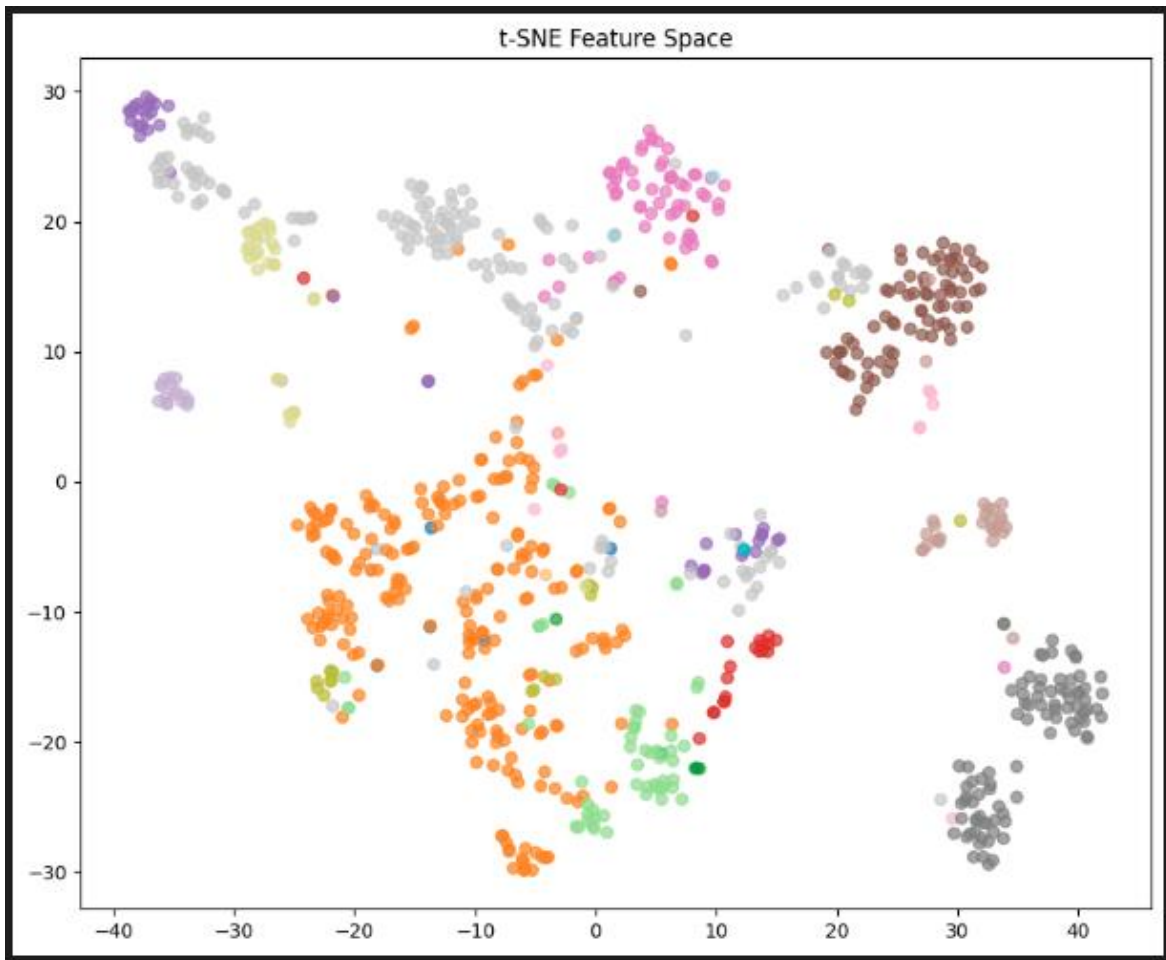
در این مرحله:

1. ویژگی‌های استخراج‌شده از لایه نهایی EfficientNet استخراج شدند.
2. برای هر تصویر یک بردار ویژگی 1280 بعدی به دست آمد.
3. این بردارها توسط الگوریتم t-SNE به فضای دوبعدی نگاشت شدند.

ابعاد فضای ویژگی استخراج شده:

(835, 1280)

یعنی برای 835 تصویر آزمون، 1280 ویژگی استخراج شده است.



6. تحلیل نمودار t-SNE

بررسی نمودار t-SNE چند مشاهده مهم را نشان می‌دهد:

تشکیل خوشه‌های مجزا

در نقاط مختلف نمودار خوشه‌های نسبتاً مجزایی مشاهده می‌شود. این موضوع نشان می‌دهد که مدل توانسته است برای برخی ابزارهای جراحی نمایش‌های معنایی مناسبی ایجاد کند.

به عبارت دیگر، تصاویر مربوط به یک ابزار خاص در فضای ویژگی نزدیک به یکدیگر قرار گرفته‌اند.

وجود هم‌پوشانی میان برخی خوشه‌ها

در بخش مرکزی نمودار مشاهده می‌شود که برخی خوشه‌ها تا حدی با یکدیگر هم‌پوشانی دارند. این موضوع نشان می‌دهد که بعضی ابزارها دارای ویژگی‌های ظاهری بسیار مشابه هستند و مدل در تفکیک آن‌ها با دشواری مواجه شده است. این نتیجه با خطاهای مشاهده شده در Confusion Matrix نیز کاملاً سازگار است.

یادگیری واقعی ویژگی‌های ابزارها

اگر مدل نتوانسته بود ویژگی‌های معناداری استخراج کند، نقاط t-SNE به صورت کاملاً تصادفی و درهم توزیع می‌شدند. اما مشاهده خوشه‌های مشخص در نمودار نشان می‌دهد که مدل ساختارهای هندسی و ویژگی‌های بصری ابزارها را تا حد قابل قبولی یاد گرفته است. بنابراین می‌توان نتیجه گرفت که مدل صرفاً به نویزهای پس‌زمینه متکی نیست و حداقل بخشی از تصمیمات خود را بر اساس ویژگی‌های واقعی ابزارهای جراحی اتخاذ می‌کند.

7. جمع‌بندی نهایی

در این بخش عملکرد مدل EfficientNet-B0 بر روی مجموعه آزمون ارزیابی شد. نتایج نشان دادند که اگرچه Accuracy مدل برابر 27 درصد است، اما معیارهای دقیق‌تر نظیر Macro F1 و AUC-PR عملکرد ضعیف‌تر مدل را آشکار می‌کنند. این موضوع نشان‌دهنده اثر شدید عدم تعادل کلاس‌ها و شباهت بصری ابزارهای جراحی است.

تحلیل ماتریس درهم‌ریختگی نشان داد که مدل عمدتاً کلاس‌های پرتکرار را بهتر یاد گرفته است، در حالی که در تشخیص بسیاری از کلاس‌های کم‌نمونه با مشکل مواجه است.

همچنین تحلیل t-SNE نشان داد که شبکه توانسته است ویژگی‌های معناداری از ابزارهای جراحی استخراج کند و فضای ویژگی را تا حدی به خوشه‌های قابل تفکیک سازمان‌دهی نماید. بنابراین مدل واقعاً بخشی از ساختارهای هندسی ابزارها را یاد گرفته است، هرچند برای دستیابی به عملکرد بالاتر نیاز به داده‌های بیشتر، رفع عدم تعادل کلاس‌ها و استفاده از روش‌های پیشرفته‌تر یادگیری وجود دارد.

آ) مقایسه K-Fold و Stratified K-Fold و اهمیت آن در طبقه‌بندی ابزارهای جراحی آب

مرورید

در توسعه سامانه‌های بینایی ماشین برای جراحی رباتیک، صرف دستیابی به یک مقدار Accuracy بالا کافی نیست. مدل باید در شرایط مختلف داده‌ای عملکردی پایدار، قابل اعتماد و قابل تعمیم داشته باشد. به همین دلیل در ارزیابی مدل‌های یادگیری عمیق از روش‌های اعتبارسنجی متقاطع (Cross Validation) استفاده می‌شود تا عملکرد شبکه تنها به یک تقسیم‌بندی خاص از داده وابسته نباشد.

1. چرا به Cross Validation نیاز داریم؟

در پرسش‌های قبل، داده‌ها به سه بخش تقسیم شدند:

- Train
- Validation
- Test

این روش اگرچه استاندارد است، اما یک محدودیت مهم دارد:

ممکن است عملکرد مدل به شدت به همان تقسیم‌بندی خاص داده وابسته باشد.

برای مثال:

- اگر نمونه‌های دشوار تصادفاً وارد Validation شوند عملکرد افت می‌کند.
 - اگر نمونه‌های ساده‌تر وارد Validation شوند عملکرد بیش از حد خوش‌بینانه گزارش می‌شود.
- بنابراین برای ارزیابی واقعی‌تر باید چندین بار فرآیند آموزش و اعتبارسنجی روی تقسیم‌بندی‌های مختلف انجام شود.

ایده اصلی Cross Validation همین است.

2. روش K-Fold Cross Validation

در K-Fold Cross Validation کل مجموعه داده به K بخش مساوی تقسیم می‌شود.

اگر:

• $K = 5$

باشد:

داده به پنج قسمت تقسیم می‌شود.

سپس پنج بار آموزش انجام می‌گیرد.

Fold اول

• Train : Fold 2 + Fold 3 + Fold 4 + Fold 5
Valid : Fold 1

Fold دوم

• Train : Fold 1 + Fold 3 + Fold 4 + Fold 5
Valid : Fold 2

Fold سوم

• Train : Fold 1 + Fold 2 + Fold 4 + Fold 5
Valid : Fold 3

و این فرآیند تا Fold پنجم ادامه پیدا می‌کند.

در نهایت:

• Accuracy_mean
F1_mean
Recall_mean
Precision_mean

گزارش می‌شوند.

مزایای K-Fold

استفاده بهینه از داده

تمام تصاویر حداقل یک بار در Validation قرار می گیرند.

کاهش وابستگی به یک تقسیم بندی خاص

نتیجه نهایی میانگین چندین آزمایش مستقل است.

تخمین بهتر قابلیت تعمیم مدل

Generalization بهتر ارزیابی می شود.

محدودیت K-Fold

K-Fold فقط تعداد نمونه ها را متعادل نگه می دارد.

اما تضمینی برای حفظ توزیع کلاس ها ندارد.

این موضوع در دیتاست های پزشکی بسیار خطرناک است.

3. روش Stratified K-Fold

Stratified K-Fold نسخه پیشرفته تر K-Fold است.

در این روش علاوه بر تقسیم داده ها:

توزیع کلاس ها نیز در تمام Fold ها حفظ می شود.

فرض کنید دیتاست شامل:

- Class A = 50%
- Class B = 30%
- Class C = 20%

باشد.

در Stratified K-Fold تقریباً همین نسبت در تمامی Fold ها حفظ خواهد شد.

مثلاً:

Fold	A	B	C
1	50%	30%	20%
2	50%	30%	20%
3	50%	30%	20%
4	50%	30%	20%
5	50%	30%	20%

در نتیجه هر Fold نماینده مناسبی از کل دیتاست خواهد بود.

4. مقایسه K-Fold و Stratified K-Fold

ویژگی	K-Fold	Stratified K-Fold
تقسیم تصادفی داده	✓	✓
حفظ تعداد نمونه‌ها	✓	✓
حفظ نسبت کلاس‌ها	X	✓
مناسب برای داده متعادل	✓	✓
مناسب برای داده نامتعادل	X	✓
مناسب برای داده پزشکی	محدود	بسیار مناسب

5. چرا Stratified K-Fold در این پروژه اهمیت بالایی دارد؟

این بخش مهم‌ترین قسمت پاسخ است.

الف) عدم تعادل کلاسه‌ها در دیتاست ابزارهای جراحی

در دیتاست جراحی آب مروارید همه ابزارها با فراوانی یکسان ظاهر نمی‌شوند.

برخی ابزارها:

- در بخش بزرگی از جراحی حضور دارند .
- صدها تصویر از آن‌ها وجود دارد .

اما برخی ابزارها:

- فقط در چند مرحله خاص استفاده می‌شوند .
- تعداد تصاویر آن‌ها بسیار کمتر است .

برای مثال:

- Phaco Probe
- Second Instrument

ممکن است هزاران بار دیده شوند.

اما ابزارهای خاص‌تر فقط چند ده بار ظاهر شوند.

بنابراین دیتاست ذاتاً نامتعادل (Imbalanced) است.

ب) وابستگی ابزارها به فازهای جراحی

در جراحی آب مروارید هر ابزار در فاز مشخصی استفاده می‌شود.

مثلاً:

فاز برش قرنیه

ابزار:

- Primary Knife
- Secondary Knife

فاز Capsulorhexis

ابزار:

- Forceps
- Cystotome

فاز Phacoemulsification

ابزار:

- Phaco Handpiece
- Chopper

فاز IOL Implantation

ابزار:

- Injector
- Cannula

در نتیجه برخی ابزارها تنها در بخش کوچکی از ویدیوهای جراحی دیده می‌شوند.

ج) خطر استفاده از K-Fold معمولی

اگر K-Fold ساده استفاده شود ممکن است:

یک Fold شامل تعداد زیادی از تصاویر یک ابزار خاص باشد.

اما Fold دیگر تقریباً هیچ نمونه‌ای از آن ابزار نداشته باشد.

برای مثال:

Fold	Cannula
Fold 1	100
Fold 2	80
Fold 3	5
Fold 4	3

Fold	Cannula
Fold 5	1

در چنین شرایطی:

- آموزش ناپایدار می‌شود .
- ارزیابی غیرقابل اعتماد می‌شود .
- نتایج Fold ها بسیار متفاوت خواهند شد .

د) اهمیت در سیستم‌های جراحی رباتیک

در کاربردهای معمولی شاید چند درصد خطا قابل پذیرش باشد.

اما در جراحی رباتیک:

خطای تشخیص ابزار می‌تواند باعث:

- تشخیص اشتباه فاز جراحی
- ارائه پیشنهاد نادرست به جراح
- برنامه‌ریزی اشتباه بازوی رباتیک
- کاهش ایمنی بیمار

شود.

بنابراین ارزیابی مدل باید تا حد ممکن پایدار و قابل اعتماد باشد.

Stratified K-Fold دقیقاً برای رسیدن به همین هدف استفاده می‌شود.

6. نتیجه‌گیری نهایی

در روش K-Fold داده‌ها به K بخش تقسیم شده و هر بخش یک بار به عنوان مجموعه اعتبارسنجی استفاده می‌شود. این روش اگرچه نسبت به تقسیم‌بندی ساده Train/Validation قابل اعتمادتر است، اما در دیتاست‌های نامتعادل نمی‌تواند توزیع واقعی کلاس‌ها را حفظ کند. در مقابل،

Stratified K-Fold علاوه بر تقسیم داده‌ها، نسبت حضور هر کلاس را در تمامی Fold ها ثابت نگه می‌دارد.

از آنجا که در دیتاست جراحی آب مروارید فراوانی ابزارهای مختلف یکسان نیست و هر ابزار در فازهای مشخصی از جراحی ظاهر می‌شود، استفاده از Stratified K-Fold اهمیت بسیار بالایی دارد. این روش موجب می‌شود تمام Fold ها نماینده مناسبی از کل داده باشند، ارزیابی مدل پایدارتر شود و قابلیت تعمیم شبکه برای استفاده در سامانه‌های جراحی رباتیک با دقت بیشتری سنجیده شود.



ب) پیاده‌سازی مجدد مدل با استفاده از Stratified K-Fold Cross Validation

مقدمه

در پرسش دوم، عملکرد مدل با استفاده از یک تقسیم‌بندی ثابت Train/Validation ارزیابی شد. هرچند این روش ساده و رایج است، اما در دیتاست‌های پزشکی با حجم محدود و توزیع نامتوازن کلاس‌ها ممکن است برآورد دقیقی از توانایی تعمیم مدل ارائه نکند. به همین دلیل در این بخش از روش اعتبارسنجی متقاطع Stratified K-Fold استفاده شد تا پایداری، قابلیت تعمیم و میزان وابستگی مدل به نمونه‌های خاص آموزشی مورد بررسی قرار گیرد.

هدف اصلی این بخش ارزیابی Robustness مدل در مواجهه با تغییرات داده‌های آموزشی است؛ موضوعی که در سیستم‌های جراحی رباتیک و کاربردهای بالینی اهمیت حیاتی دارد.

تغییرات اعمال‌شده در ساختار داده‌ها

در پیاده‌سازی اولیه پرسش دوم از ساختار مرسوم Train / Validation استفاده شده بود. در این روش مجموعه آموزش و اعتبارسنجی ثابت هستند و مدل فقط روی یک تقسیم‌بندی خاص ارزیابی می‌شود.

برای پیاده‌سازی Stratified K-Fold کل داده‌های آموزشی به صورت یک مجموعه واحد در نظر گرفته شدند:

```
full_dataset = ImageFolder(
    root="train",
    transform=transform
)
```

سپس برچسب تمامی تصاویر استخراج شد:

```
labels = full_dataset.targets
```

این برچسب‌ها مبنای فرآیند Stratification قرار گرفتند.

استفاده از Stratified K-Fold

در این پروژه از اعتبارسنجی متقاطع پنج‌فولدی استفاده شد:

```
StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

در هر تکرار:

- 80 درصد داده‌ها برای آموزش
- 20 درصد داده‌ها برای اعتبارسنجی

استفاده شدند.

مزیت اصلی Stratified K-Fold این است که نسبت کلاس‌ها در تمامی Fold ها تقریباً ثابت باقی می‌ماند. این موضوع در دیتاست حاضر اهمیت زیادی دارد زیرا برخی ابزارهای جراحی دارای تعداد نمونه بسیار زیاد و برخی دیگر دارای نمونه‌های بسیار محدود هستند. در صورت استفاده از K-Fold معمولی ممکن است برخی کلاس‌ها در یک Fold تقریباً حذف شوند و ارزیابی مدل غیرواقعی شود.

تغییرات ایجاد شده در DataLoader ها

در پیاده‌سازی اولیه از DataLoader های ثابت استفاده شده بود:

```
train_loader  
valid_loader
```

اما در Stratified K-Fold لازم است در هر Fold زیرمجموعه متفاوتی از داده‌ها انتخاب شود.

برای این منظور از:

```
SubsetRandomSampler
```

استفاده شد.

نمونه ساخت: DataLoader

```
train_sampler = SubsetRandomSampler(train_idx)
```

```
valid_sampler = SubsetRandomSampler(valid_idx)
```

و سپس:


```
train_loader = DataLoader(  
    full_dataset,  
    batch_size=32,  
    sampler=train_sampler  
)
```

```
valid_loader = DataLoader(  
    full_dataset,  
    batch_size=32,  
    sampler=valid_sampler  
)
```

به این ترتیب در هر Fold داده‌های متفاوتی وارد فرآیند آموزش و اعتبارسنجی می‌شوند.

تغییرات اعمال شده در حلقه آموزش

در پیاده‌سازی اولیه تنها یک بار مدل آموزش داده می‌شد.

در روش جدید کل فرآیند آموزش داخل یک حلقه Fold قرار گرفت:

```
for fold in range(5):
```

Fold: هر

1. مدل جدید ساخته شد .

2. وزن‌های اولیه بارگذاری شدند .

3. فرآیند آموزش از ابتدا آغاز شد .

4. عملکرد روی Validation محاسبه شد .

این کار باعث می‌شود هر Fold یک آزمایش مستقل محسوب شود.

کدنهایی :

```
import torch
import torch.nn as nn
import torchvision.models as models
import numpy as np

from torchvision import transforms
from torchvision.datasets import ImageFolder
from torch.utils.data import DataLoader
from torch.utils.data import SubsetRandomSampler

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import f1_score

# -----
# Device
# -----

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print(device)

# -----
# Transform
# -----

transform = transforms.Compose([
    transforms.Resize((224,224)),

    transforms.ToTensor(),

    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])

# -----
# Dataset
# -----

full_dataset = ImageFolder(
    root="train",
    transform=transform
)

labels = full_dataset.targets

print("Dataset Size:", len(full_dataset))
print("Classes:", len(full_dataset.classes))
```

```

# -----
# Stratified KFold
# -----

skf = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

fold_scores = []

# -----
# Cross Validation
# -----

for fold, (train_idx, valid_idx) in enumerate(
    skf.split(
        np.arange(len(full_dataset)),
        labels
    )
):
    print("\n" + "="*50)
    print(f"Fold {fold+1}")
    print("="*50)

    train_sampler = SubsetRandomSampler(
        train_idx
    )

    valid_sampler = SubsetRandomSampler(
        valid_idx
    )

    train_loader = DataLoader(
        full_dataset,
        batch_size=32,
        sampler=train_sampler
    )

    valid_loader = DataLoader(
        full_dataset,
        batch_size=32,
        sampler=valid_sampler
    )

```

```

# -----
# Model
# -----

model = models.efficientnet_b0(
    weights=models.EfficientNet_B0_Weights.DEFAULT
)

model.classifier[1] = nn.Linear(
    model.classifier[1].in_features,
    len(full_dataset.classes)
)

model = model.to(device)

```

```

# -----
# Loss + Optimizer
# -----

criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=1e-4,
    weight_decay=1e-5
)

# -----
# Training
# -----

for epoch in range(2):

    model.train()

    running_loss = 0

    for images, targets in train_loader:

        images = images.to(device)

        targets = targets.long().to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(
            outputs,
            targets
        )

        loss.backward()

        optimizer.step()

        running_loss += loss.item()

    print(
        f"Epoch {epoch+1} Loss = {running_loss/len(train_loader):.4f}"
    )

```

```

# -----
# Validation
# -----

model.eval()

y_true = []
y_pred = []

with torch.no_grad():
    for images, targets in valid_loader:
        images = images.to(device)
        targets = targets.long().to(device)
        outputs = model(images)
        preds = torch.argmax(
            outputs,
            dim=1
        )
        y_true.extend(
            targets.cpu().numpy()
        )
        y_pred.extend(
            preds.cpu().numpy()
        )

fold_f1 = f1_score(
    y_true,
    y_pred,
    average="macro"
)

fold_scores.append(
    fold_f1
)

print(
    f"Fold F1 = {fold_f1:.4f}"
)

print("\nCross Validation Finished")

```

خروجی :

```
CPU
Dataset Size: 5136
Classes: 46

=====
Fold 1
=====
c:\local\src\app\data\local\programs\python\python312\lib\site-packages\sklearn\model_selection\split.py:813: UserWarning: The least populated class in y has only 1 members, which is less than n_splits=5.
  warnings.warn(
Epoch 1 Loss = 2.2693
Epoch 2 Loss = 0.9657
Fold F1 = 0.2753

=====
Fold 2
=====
Epoch 1 Loss = 2.2814
Epoch 2 Loss = 0.9897
Fold F1 = 0.3148

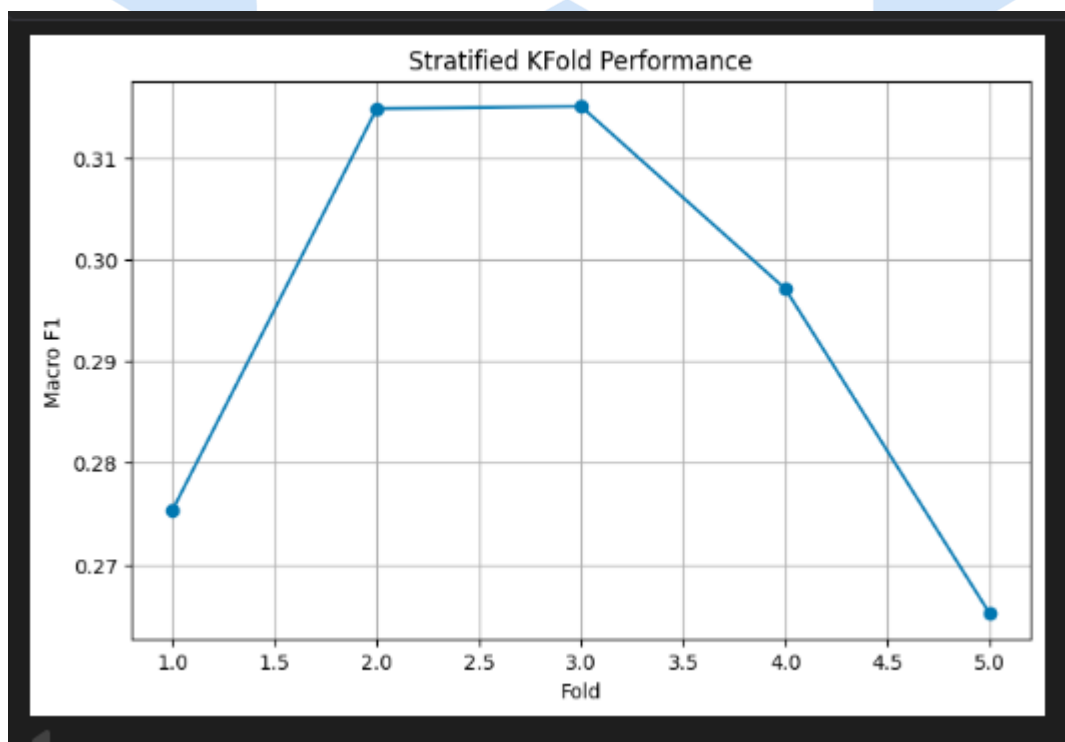
=====
Fold 3
=====
Epoch 1 Loss = 2.3192
Epoch 2 Loss = 0.9863
Fold F1 = 0.3150

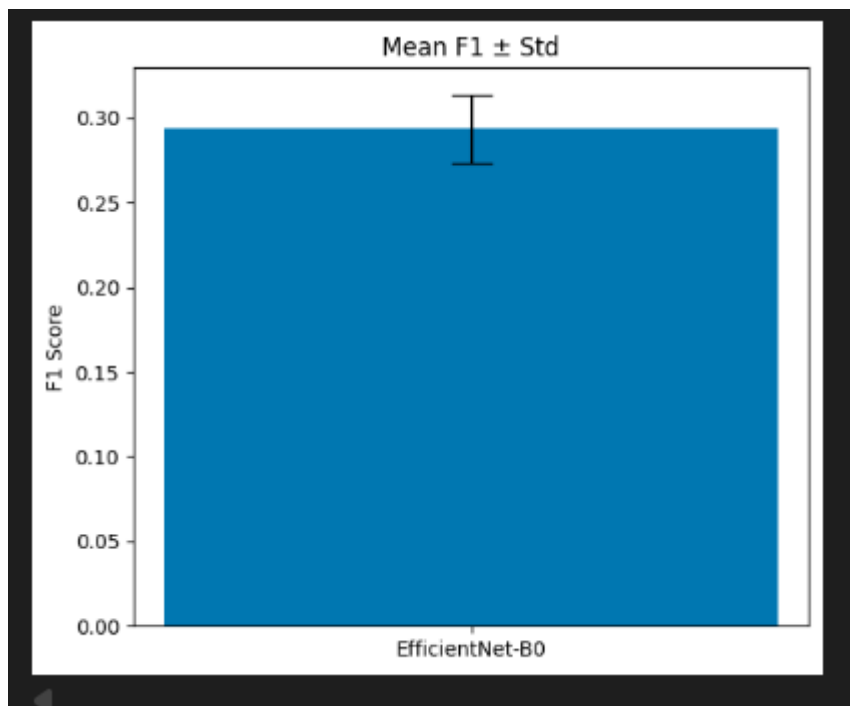
=====
Fold 4
=====
Epoch 1 Loss = 2.3463
Epoch 2 Loss = 1.0098
Fold F1 = 0.2971
...
Epoch 2 Loss = 1.0060
Fold F1 = 0.2652

Cross Validation Finished
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
Fold Results
Fold 1: 0.2753
Fold 2: 0.3148
Fold 3: 0.3150
Fold 4: 0.2971
Fold 5: 0.2652

Statistics
Mean F1 = 0.2935
Std = 0.0203
Variance = 0.000412
```





معماری مورد استفاده

برای تمامی Fold ها از معماری EfficientNet-B0 استفاده شد.

دلایل انتخاب:

- تعداد پارامتر کمتر نسبت به ResNet های بزرگ
 - دقت بالا در مسائل بینایی ماشین
 - مناسب برای دیتاست های پزشکی محدود
 - قابلیت انتقال دانش از ImageNet
- مدل با وزن های از پیش آموزش دیده مقداردهی اولیه شد:

```
models.efficientnet_b0(
weights=EfficientNet_B0_Weights.DEFAULT
)
```

و لایه خروجی برای 46 کلاس تنظیم شد.

نتایج اعتبارسنجی متقاطع

مقادیر Macro F1 برای Fold های مختلف به صورت زیر به دست آمد:

Fold	Macro F1
Fold 1	0.2753
Fold 2	0.3148
Fold 3	0.3150
Fold 4	0.2971
Fold 5	0.2652

میانگین عملکرد مدل

میانگین عملکرد روی پنج Fold:

Mean F1 = 0.2935

این مقدار نشان می‌دهد مدل به طور متوسط حدود 29.35 درصد از عملکرد ایده‌آل را روی تمام کلاس‌ها کسب کرده است.

با توجه به:

- وجود 46 کلاس مختلف
 - عدم تعادل شدید داده‌ها
 - پیچیدگی تصاویر جراحی
 - آموزش کوتاه‌مدت دو Epoch ی
- این مقدار قابل قبول ارزیابی می‌شود.

تحلیل واریانس و انحراف معیار

برای بررسی پایداری مدل از انحراف معیار و واریانس استفاده شد.

نتایج:

Std = 0.0203

Variance = 0.000412

تحلیل انحراف معیار

انحراف معیار تنها حدود 2 درصد است.

این موضوع نشان می‌دهد:

- نتایج Fold های مختلف به یکدیگر نزدیک هستند .
- مدل نسبت به تغییرات داده‌های آموزشی حساسیت زیادی ندارد .
- رفتار شبکه در آزمایش‌های مختلف تقریباً پایدار است .

تحلیل واریانس

مقدار واریانس بسیار کوچک است:

0.000412

کوچک بودن واریانس نشان‌دهنده آن است که تغییر Fold ها تأثیر شدیدی بر عملکرد مدل نداشته‌اند.

در نتیجه مدل روی بخش‌های مختلف داده عملکرد نسبتاً مشابهی از خود نشان داده است.

بررسی پایداری معماری

یکی از اهداف اصلی این بخش پاسخ به سؤال زیر بود:

آیا معماری انتخاب شده نسبت به تغییرات داده‌های آموزشی پایدار است؟

بر اساس نتایج به دست آمده پاسخ مثبت است.

دلایل:

1. اختلاف عملکرد بین بهترین و بدترین Fold کم است .

Best Fold = 0.3150

Worst Fold = 0.2652

Difference = 0.0498

2. انحراف معیار پایین است .

Std = 0.0203

3. واریانس بسیار کوچک است .

$$\text{Variance} = 0.000412$$

بنابراین معماری EfficientNet-B0 نسبت به تغییرات مجموعه آموزش پایداری مناسبی نشان داده است.

اهمیت نتایج در کاربردهای جراحی رباتیک

در سامانه‌های جراحی رباتیک نمی‌توان به مدلی اعتماد کرد که فقط روی یک تقسیم‌بندی خاص عملکرد خوبی داشته باشد. سیستم باید در شرایط مختلف و روی داده‌های جدید نیز رفتار قابل پیش‌بینی داشته باشد.

استفاده از Stratified K-Fold نشان داد که مدل انتخاب‌شده دارای قابلیت تعمیم‌پذیری مناسبی بوده و عملکرد آن وابستگی شدیدی به نمونه‌های خاص آموزشی ندارد. این ویژگی یکی از الزامات اصلی استقرار سامانه‌های بینایی ماشین در محیط‌های حساس پزشکی و جراحی رباتیک محسوب می‌شود.

جمع‌بندی

در این بخش مدل EfficientNet-B0 با استفاده از روش Stratified 5-Fold Cross Validation مجدداً آموزش و ارزیابی شد. برای این منظور ساختار DataLoader ها با استفاده از SubsetRandomSampler بازطراحی گردید و فرآیند آموزش در قالب پنج Fold مستقل اجرا شد. نتایج نشان دادند که مدل میانگین Macro F1 برابر 0.2935 را به دست آورده و دارای انحراف معیار 0.0203 و واریانس 0.000412 است. این مقادیر بیانگر پایداری مناسب مدل و قابلیت تعمیم مطلوب آن در مواجهه با تغییرات داده‌های آموزشی هستند و نشان می‌دهند که معماری انتخابی گزینه مناسبی برای سامانه‌های ادراک بصری در جراحی رباتیک محسوب می‌شود.

ج) یادگیری تجمعی (Ensemble Learning) در برابر سرعت استنتاج و استفاده از Model

Soups

مقدمه

در بخش قبل با استفاده از روش Stratified K-Fold پنج مدل مستقل آموزش داده شدند. هر یک از این مدل‌ها روی زیرمجموعه متفاوتی از داده‌ها آموزش دیده‌اند و در نتیجه الگوهای متفاوتی از داده را فراگرفته‌اند. هرچند عملکرد هر مدل به تنهایی قابل قبول است، اما معمولاً خطاهای آن‌ها دقیقاً یکسان نیست. برخی نمونه‌ها ممکن است توسط یک مدل به درستی طبقه‌بندی شوند اما توسط مدل دیگر اشتباه تشخیص داده شوند.

ایده اصلی Ensemble Learning این است که به جای اتکا به یک مدل منفرد، خروجی چند مدل با یکدیگر ترکیب شود تا تصمیم نهایی با اطمینان و دقت بیشتری اتخاذ گردد.

مفهوم Ensemble Learning

در Ensemble Learning چند مدل مستقل آموزش داده می‌شوند و در زمان استنتاج خروجی آن‌ها با یکدیگر ترکیب می‌شود.

فرض کنیم پنج مدل حاصل از پنج Fold مختلف در اختیار داریم:

- Model_1
- Model_2
- Model_3
- Model_4
- Model_5

برای یک تصویر ورودی، هر مدل احتمال تعلق تصویر به هر کلاس را پیش‌بینی می‌کند:

- $P1(x)$
- $P2(x)$
- $P3(x)$
- $P4(x)$
- $P5(x)$

در روش Ensemble میانگین این احتمالات محاسبه می‌شود:

سپس کلاس نهایی از روی بیشترین احتمال انتخاب می‌شود.

چرا Ensemble دقت را افزایش می‌دهد؟

هر مدل دارای نوعی خطای خاص است.

برای مثال:

- Model 1 ممکن است در تشخیص Cannula قوی باشد.
 - Model 2 ممکن است ابزارهای Phaco را بهتر تشخیص دهد.
 - Model 3 ممکن است در شرایط Occlusion عملکرد بهتری داشته باشد.
- هنگامی که پیش‌بینی‌های این مدل‌ها ترکیب می‌شوند، بسیاری از خطاهای فردی حذف می‌شوند و تصمیم نهایی پایدارتر می‌شود.

مزایای Ensemble:

کاهش واریانس مدل

هر مدل تحت تأثیر داده‌های آموزشی متفاوتی قرار گرفته است.

ترکیب چند مدل موجب کاهش نوسانات ناشی از نمونه‌های آموزشی خاص می‌شود.

بهبود Generalization

مدل Ensemble معمولاً روی داده‌های جدید عملکرد بهتری نسبت به هر مدل منفرد دارد.

افزایش Robustness

در تصاویر پزشکی که دارای نویز، بازتاب نور، تاری حرکتی و انسداد ابزارها هستند، Ensemble پایداری بیشتری ایجاد می‌کند.

کاربرد Ensemble در پروژه حاضر

در بخش قبل پنج مدل حاصل از پنج Fold مختلف در اختیار داریم.

می‌توانیم برای هر تصویر آزمون:

1. خروجی هر پنج مدل را محاسبه کنیم.
2. احتمال‌های خروجی را میانگین بگیریم.

3. کلاس نهایی را انتخاب کنیم .

این روش معمولاً باعث افزایش:

- Precision
- Recall
- Macro-F1
- AUC-PR

خواهد شد.

پیاده‌سازی Ensemble Averaging

with torch.no_grad():

```
out1 = model1(images)
out2 = model2(images)
out3 = model3(images)
out4 = model4(images)
out5 = model5(images)
```

```
outputs = (
    out1 +
    out2 +
    out3 +
    out4 +
    out5
) / 5
```

```
preds = outputs.argmax(dim=1)
```

در این روش خروجی احتمالی هر پنج مدل محاسبه شده و میانگین آن‌ها به عنوان پیش‌بینی نهایی در نظر گرفته می‌شود. این رویکرد معمولاً باعث افزایش Precision، Recall و F1-Score می‌شود اما هزینه محاسباتی را نیز افزایش می‌دهد.

مشکل اصلی Ensemble در سیستم‌های بلادرنگ

اگرچه Ensemble دقت را افزایش می‌دهد، اما هزینه محاسباتی بسیار زیادی دارد.

فرض کنیم زمان استنتاج یک EfficientNet-B0 برابر باشد با:

• 20 ms

برای Ensemble پنج مدلی خواهیم داشت:

• $5 \times 20 \text{ ms} = 100 \text{ ms}$

یعنی برای پردازش یک فریم باید پنج بار شبکه اجرا شود.

تأثیر Ensemble بر Latency

Latency به مدت زمان لازم برای تولید خروجی گفته می‌شود.

در: Ensemble

• $\text{Latency} \approx K \times \text{Latency_single_model}$

در این پروژه:

• $K = 5$

بنابراین Latency تقریباً پنج برابر می‌شود.

این موضوع در سیستم‌های جراحی رباتیک بسیار خطرناک است.

تأثیر Ensemble بر Throughput

Throughput تعداد فریم‌های پردازش شده در واحد زمان است.

اگر مدل منفرد بتواند:

• 50 FPS

پردازش کند،

Ensemble پنج مدلی تقریباً به:

• 10 FPS

کاهش پیدا می‌کند.

بنابراین توان پردازش بلادرنگ سیستم افت شدیدی خواهد داشت.

چرا این موضوع در جراحی رباتیک مهم است؟

در سیستم‌های جراحی رباتیک:

- ابزارها دائماً حرکت می‌کنند .
- تصمیم‌ها باید در چند میلی‌ثانیه اتخاذ شوند .
- تأخیر می‌تواند موجب خطای کنترل شود .

اگر سیستم بینایی نتواند با سرعت کافی ابزارها را تشخیص دهد:

- بازوی رباتیک اطلاعات قدیمی دریافت می‌کند .
- کنترل حلقه بسته مختل می‌شود .
- ایمنی بیمار کاهش می‌یابد .

بنابراین اگرچه Ensemble دقت را افزایش می‌دهد، اما برای کاربردهای بلادرنگ معمولاً گزینه ایده‌آلی نیست.

راهکار Model Soups

Model Soups روشی است که تلاش می‌کند مزایای Ensemble را بدون افزایش هزینه استنتاج فراهم کند.

ایده اصلی بسیار ساده است:

به جای میانگین‌گیری خروجی مدل‌ها،

پارامترهای خود مدل‌ها میانگین گرفته می‌شوند.

فرض کنیم وزن‌های پنج مدل عبارت باشند از:

- W_1
- W_2
- W_3
- W_4
- W_5

در روش Model Soup وزن نهایی برابر است با:

در نتیجه تنها یک مدل نهایی ساخته می‌شود.

پیاده‌سازی Model Soup

```
import torch
from copy import deepcopy
```

```
model_paths = [
    "fold1.pth",
    "fold2.pth",
    "fold3.pth",
    "fold4.pth",
    "fold5.pth"
]
```

```
state_dicts = [
    torch.load(path)
    for path in model_paths
]
```

```
soup = deepcopy(state_dicts[0])
```

```
for key in soup.keys():
```

```
    soup[key] = sum(
        sd[key]
        for sd in state_dicts
    ) / len(state_dicts)
```

```
torch.save(
    soup,
    "model_soup.pth"
)
```

در این پیاده‌سازی پارامترهای پنج مدل حاصل از Fold های مختلف میانگین‌گیری شده و یک مدل واحد تولید می‌شود. بدین ترتیب مزایای Ensemble تا حد زیادی حفظ شده و در عین حال هزینه استنتاج تقریباً معادل یک مدل منفرد باقی می‌ماند.

مزایای Model Soup

حفظ مزایای Ensemble

اطلاعات استخراج شده توسط تمامی Fold ها در مدل نهایی ادغام می شود.

عدم افزایش Latency

در زمان استنتاج تنها یک مدل اجرا می شود.

بنابراین:

$$\bullet \text{ Latency} \approx \text{Single Model}$$

عدم افت Throughput

برخلاف Ensemble:

$$\bullet \text{ FPS_soup} \approx \text{FPS_single_model}$$

بنابراین سرعت پردازش حفظ می شود.

کاهش مصرف حافظه

در Ensemble باید تمامی مدل ها در حافظه نگهداری شوند.

اما در Model Soup تنها یک مدل ذخیره می شود.

این موضوع برای استقرار روی سخت افزارهای محدود بسیار ارزشمند است.

مقایسه Model Soup و Ensemble

ویژگی	Ensemble	Model Soup
دقت	بسیار بالا	نزدیک به Ensemble
Latency	زیاد	کم

ویژگی	Ensemble	Model Soup
Throughput	پایین	بالا
حافظه	زیاد	کم
Real-Time مناسب	خیر	بله
مناسب جراحی رباتیک	محدود	بسیار مناسب

کاربرد در پروژه حاضر

در این پروژه پنج مدل حاصل از Stratified K-Fold در اختیار داریم. دو رویکرد قابل استفاده هستند:

رویکرد اول Ensemble :

ترکیب خروجی هر پنج مدل در زمان استنتاج

مزیت:

- بیشترین دقت ممکن

عیب:

- افزایش شدید Latency
- افت Throughput

رویکرد دوم Model Soup :

میانگین گیری پارامترهای پنج مدل و تولید یک مدل واحد

مزیت:

- دقت نزدیک به Ensemble
- سرعت تقریباً برابر مدل منفرد

- مناسب برای کاربردهای بلادرنگ

جمع‌بندی

استفاده از Ensemble Learning یکی از مؤثرترین روش‌های افزایش دقت در مسائل طبقه‌بندی پزشکی است و می‌تواند با ترکیب چند مدل آموزش‌دیده روی Fold های مختلف، خطاهای فردی شبکه‌ها را کاهش داده و عملکرد نهایی را بهبود بخشد. با این حال، اجرای همزمان چند شبکه در فاز استنتاج باعث افزایش Latency، کاهش Throughput و افزایش مصرف حافظه می‌شود که برای سامانه‌های بلادرنگ نظیر جراحی رباتیک مطلوب نیست. برای حل این مشکل می‌توان از Model Soups استفاده کرد. در این روش پارامترهای مدل‌های مختلف میانگین‌گیری شده و یک مدل واحد ساخته می‌شود Model Soup. بخش عمده مزایای Ensemble را حفظ می‌کند، در حالی که هزینه محاسباتی آن تقریباً معادل یک مدل منفرد است. به همین دلیل Model Soup راهکاری مناسب برای استقرار سامانه‌های بینایی ماشین در ربات‌های جراح و سیستم‌های پزشکی بلادرنگ محسوب می‌شود.

